

算法设计与分析

Lecture 6: Dynamic Programming (动态规划)

曹刘娟

厦门大学信息学院计算机系

caoliujuan@xmu.edu.cn



Fibonacci Sequence Revisit

- Fibonacci sequence is defined by

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}, \text{ for } n \geq 2$$

Fib1(n)

1 **if** $n \leq 1$ **then**

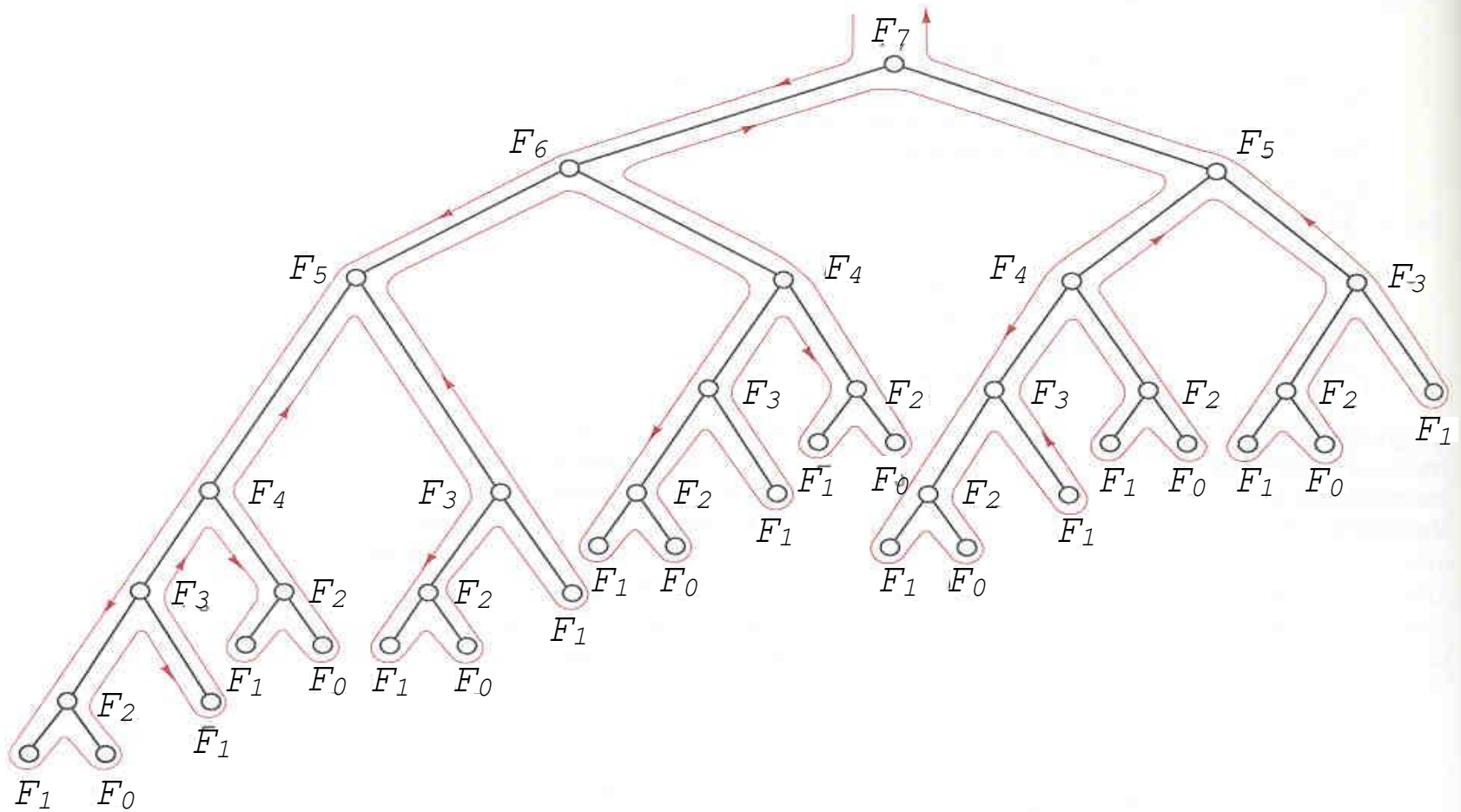
2 **return** n

3 **else**

4 **return** Fib1($n - 1$) + Fib1($n - 2$)

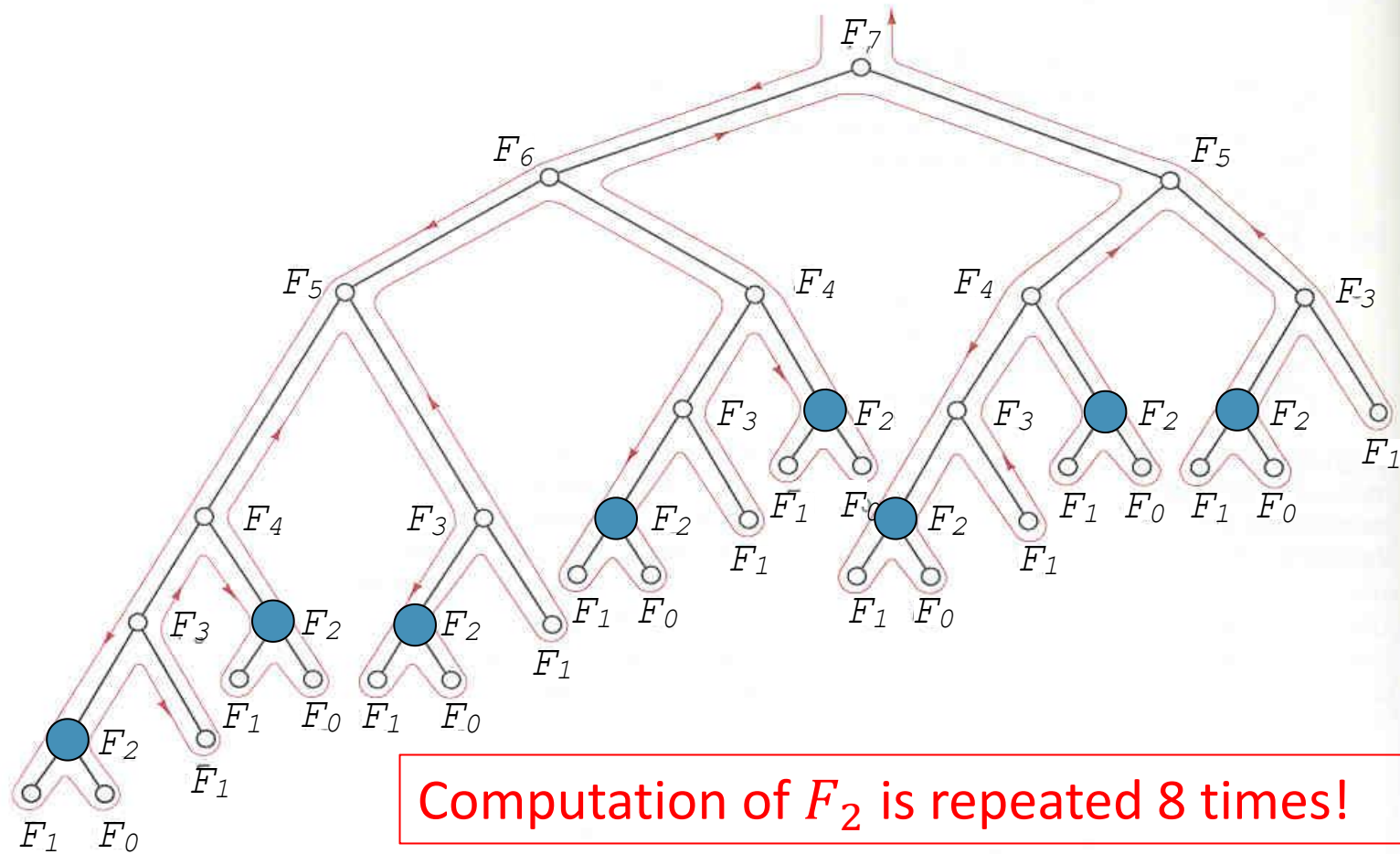


Fibonacci Sequence Revisit



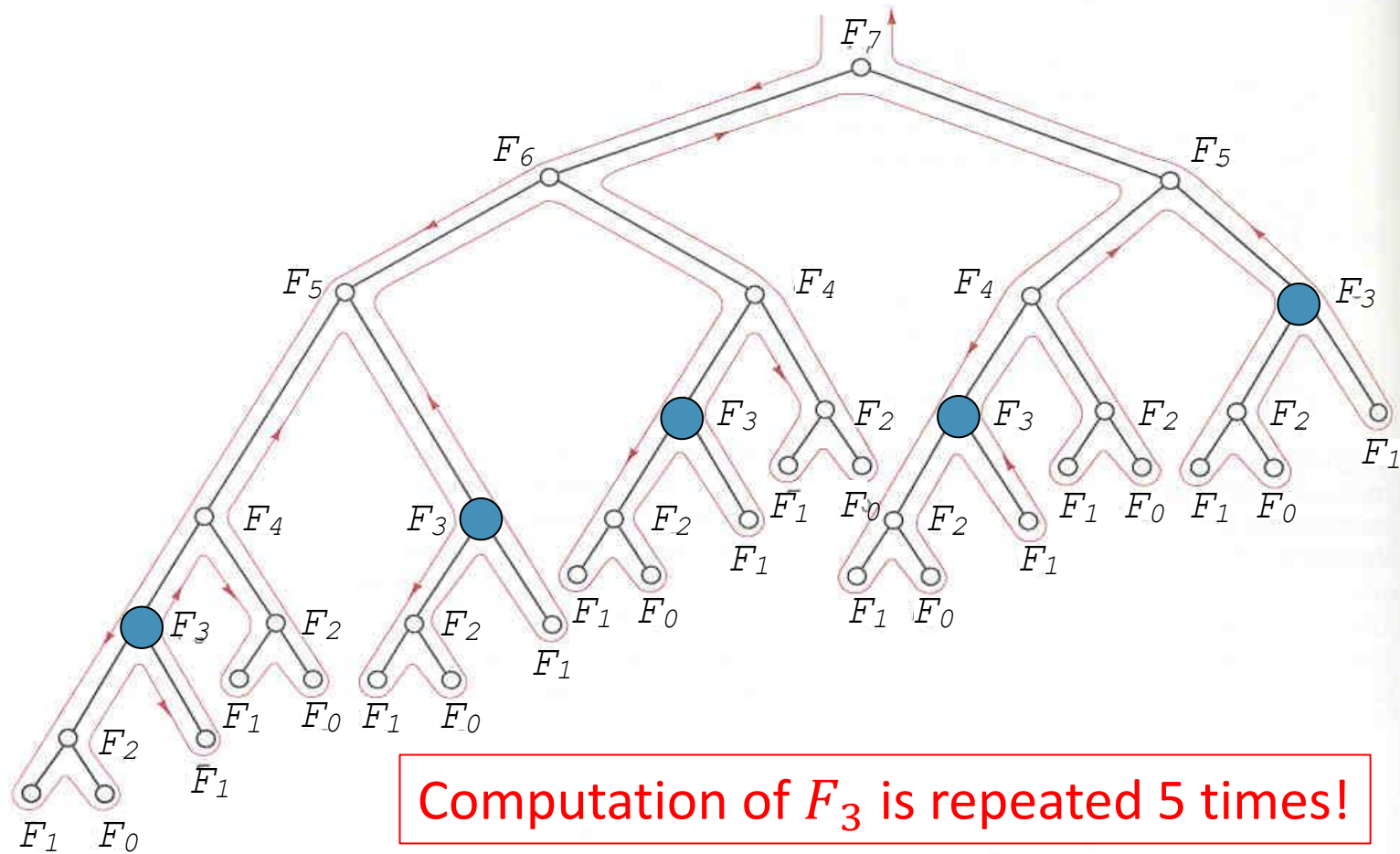


Fibonacci Sequence Revisit





Fibonacci Sequence Revisit





Fibonacci Sequence Revisit

- Idea of improvement: **record the values somewhere!**

将原有的计算过的值保存起来

- Store F_i somewhere after we have computed its value.
- Afterward, we don't need to re-compute F_i . We can retrieve its value from our memory.

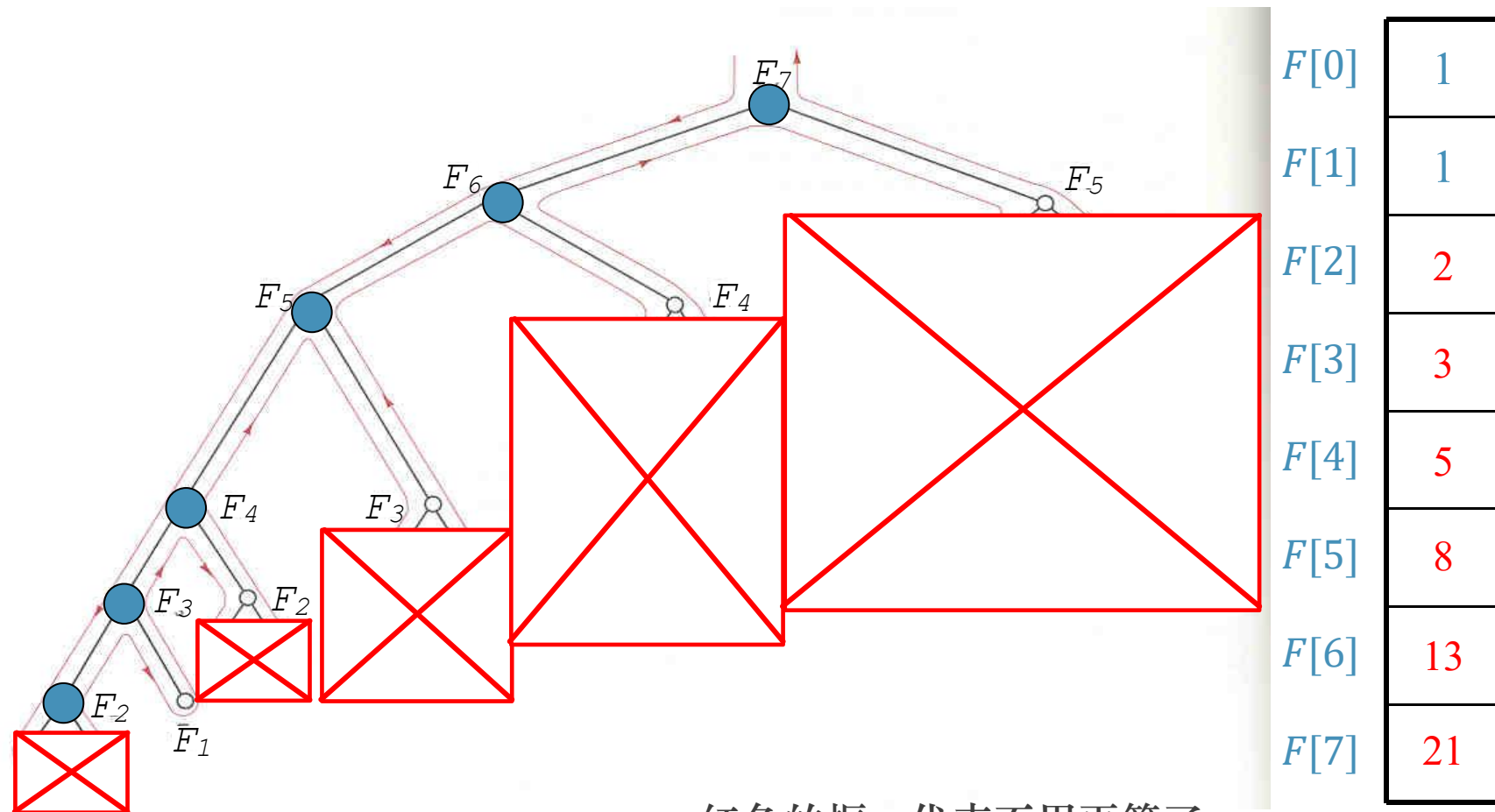
```
Fib2(n)
1  if  $F[n] < 0$  then
2     $F[n] \leftarrow \text{Fib2}(n - 1) + \text{Fib2}(n - 2)$ 
3  return  $F[n]$ 
```

```
Main()
1   $F[0] = F[1] \leftarrow 1$ 
2  for  $i \leftarrow 2$  to  $n$  do
3     $F[i] \leftarrow -1$ 
4  print  $\text{Fib2}(n)$ 
```

还是用递归，但是加了一个判断



Fibonacci Sequence Revisit



红色的框，代表不用再算了



Fibonacci Sequence Revisit

- Can we do even better?
 - Although we didn't waste time on repeated computation, we make a lot of recursive function calls. 虽然我们并没有浪费重复的计算，但是花了很多时间在递归调用上
 - Must we use recursion?
- Idea to further improve
 - Compute the values **in bottom-up fashion**. 可以自底向上的来解决
 - That is, compute F_2 (we already know $F_0 = F_1 = 1$), then F_3 , then F_4 ...

```
Fib3(n)
1   $F[0] \leftarrow 1$ 
2   $F[1] \leftarrow 1$ 
3  for  $i \leftarrow 2$  to  $n$  do
4       $F[i] \leftarrow F[i - 1] + F[i - 2]$ 
5  return  $F[n]$ 
```

This new implementation
saves lots of overhead.



Recursion vs. Dynamic Programming

■ Recursion:

- Too slow.
- Time complexity: $O(2^n)$.

```
Fib1(n)
1 if  $n \leq 1$  then
2   return  $n$ 
3 else
4   return Fib1( $n - 1$ ) +
      Fib1( $n - 2$ )
```

■ Dynamic Programming:

- Efficient!
- Time complexity: $O(n)$.

```
Fib3(n)
1  $F[0] \leftarrow 1$ 
2  $F[1] \leftarrow 1$ 
3 for  $i \leftarrow 2$  to  $n$  do
4    $F[i] \leftarrow F[i - 1] + F[i - 2]$ 
5 return  $F[n]$ 
```



动态规划算法计算步骤

- Write down a formula that relates a solution of a problem instance with those of small instances. (构造递归方程)
 - E.g. $F(n) = F(n - 1) + F(n - 2)$.
- Index the sub-problems so that they can be **stored and retrieved easily** in a table (i.e., array). (保存子问题值)
- Fill the table in some **bottom-up manner**; start filling the solution of the smallest instance. (自底向上填数组)
 - This ensures that when we **solve a particular instance**, the solutions of **all the smaller instances** that it depends are available.



Dynamic Programming

- For historical reasons, we call such methodology **Dynamic Programming (DP, 动态规划)**.
- It was developed by American applied mathematician Richard Ernest Bellman in 1950s.
- It is also called Bellman equation **in optimization field**.
- 他发明的贝尔曼方程也被广泛的应用于控制、强化学习领域



Richard Ernest Bellman
(1920-1984)



Divide-and-Conquer vs. Dynamic Programming

分治和动态规划对比分析

- **Common:** Problem is partitioned into one or more subproblem, then the solution of subproblem is combined. 和分治一样，问题是可以切分成很多小问题，然后结合起来
- Divide-and-conquer method
 1. Subproblem is independent. 在分治里面，子问题之间是**独立的**
 2. Subproblem is solved repeatedly. 子问题可能会被**重复求解**
- Dynamic programming
 1. Subproblem is not independent. 子问题之间**不是相互独立的**
 2. Subproblem is just solved once. 每个子问题**只被求解一次**
- Dynamic programming **reduces computation** by
 1. Solving subproblems in a bottom-up fashion. 以自底向上的方式来解决这个问题
 2. Storing solution to a subproblem the first time it is solved. 记录子问题的答案
 3. Looking up the solution when subproblem is met again. 当遇到同样子问题的时候，就用存储的答案就行了



Dynamic Programming

■ Key idea:

- **Top-down design**: Determine structure of optimal solutions. 自顶向下设计
- **Bottom-up solve**: Avoid repeated computation. 求解
- Dynamic programming is typically applied to **optimization problems**. It is broken into a sequence of four steps. 动态规划一般和优化问题紧密结合
 1. Characterize the **structure of an optimal solution**. 最优答案的结构
 2. Recursively define the value of an optimal solution. 递归定义最优答案的值
 3. Compute the value of an optimal solution in a **bottom-up fashion**. 由下而上构造最优答案
 4. Construct an optimal solution from computed information. 基于计算出的信息构造最优答案



Examples in This Lecture

- Assembly-line scheduling 装配线调度
- Matrix-chain multiplication 矩阵连乘
- The longest-common-subsequence problem 最长公共子序列
- 0/1 knapsack problem 01背包问题
- Optimal binary search tree 最优二分搜索树
- ...



ASSEMBLY-LINE SCHEDULING

装配线调度



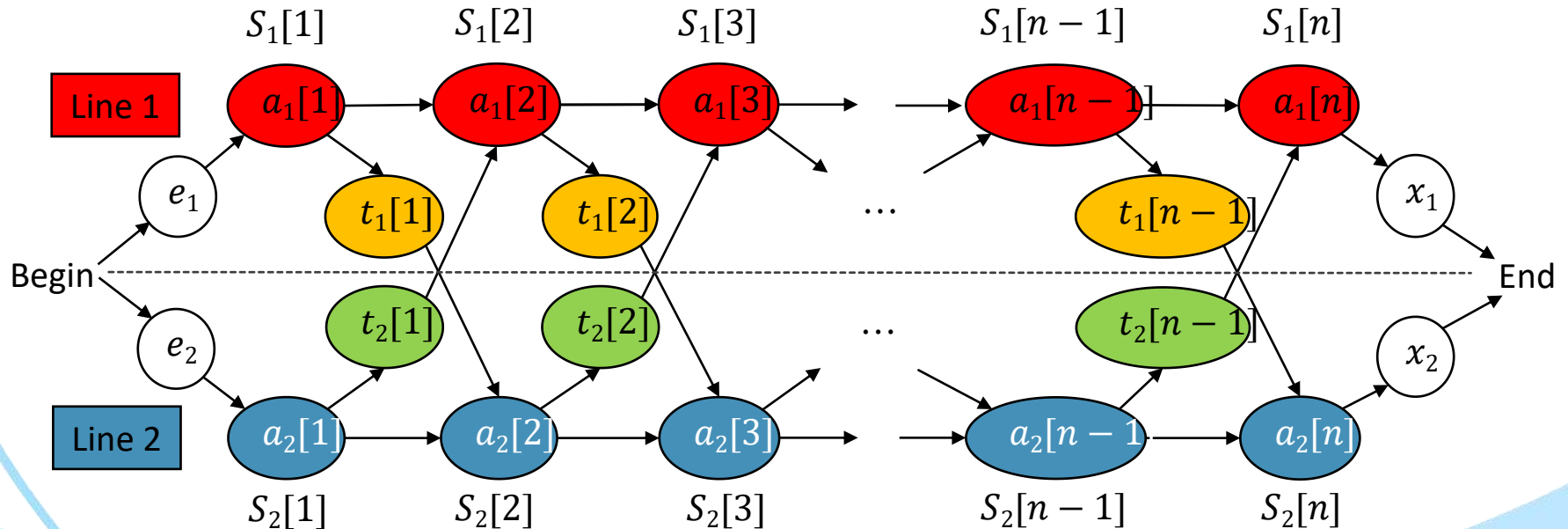
Assembly-Line Scheduling

Assembly-line scheduling (装配线调度) problem:

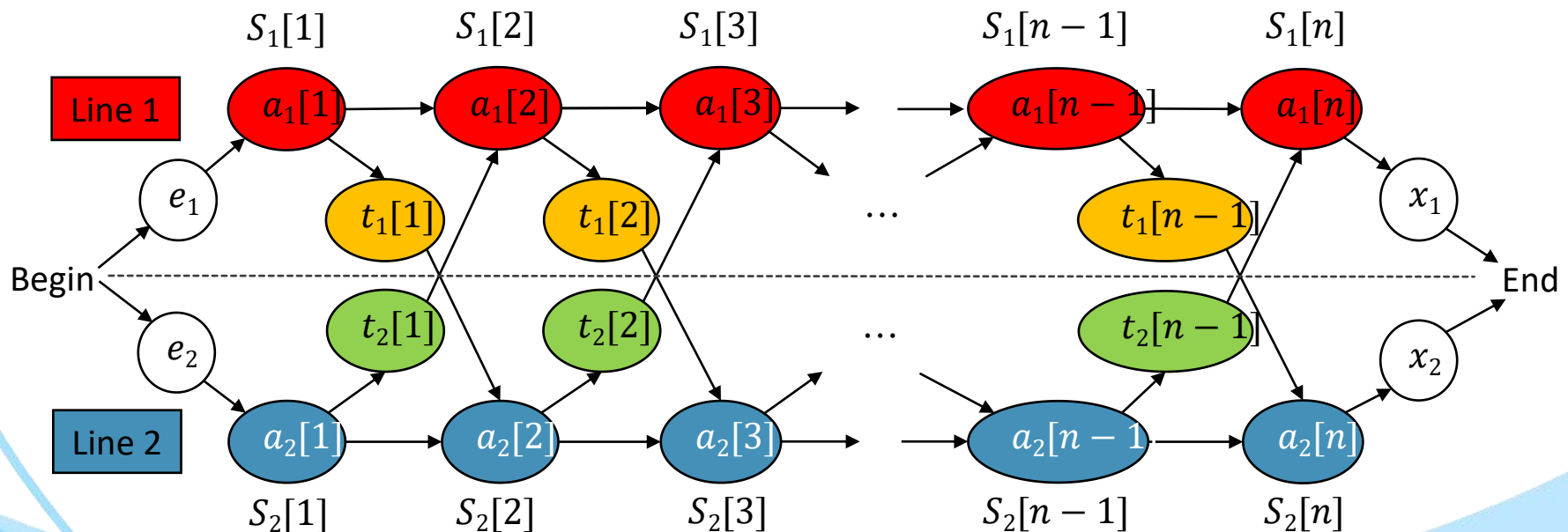
- There are **two assembly lines** (装配线), each with **n stations** (装配点).
- An automobile chassis (汽车底盘) enters the factory needs to go through **all n stations**.



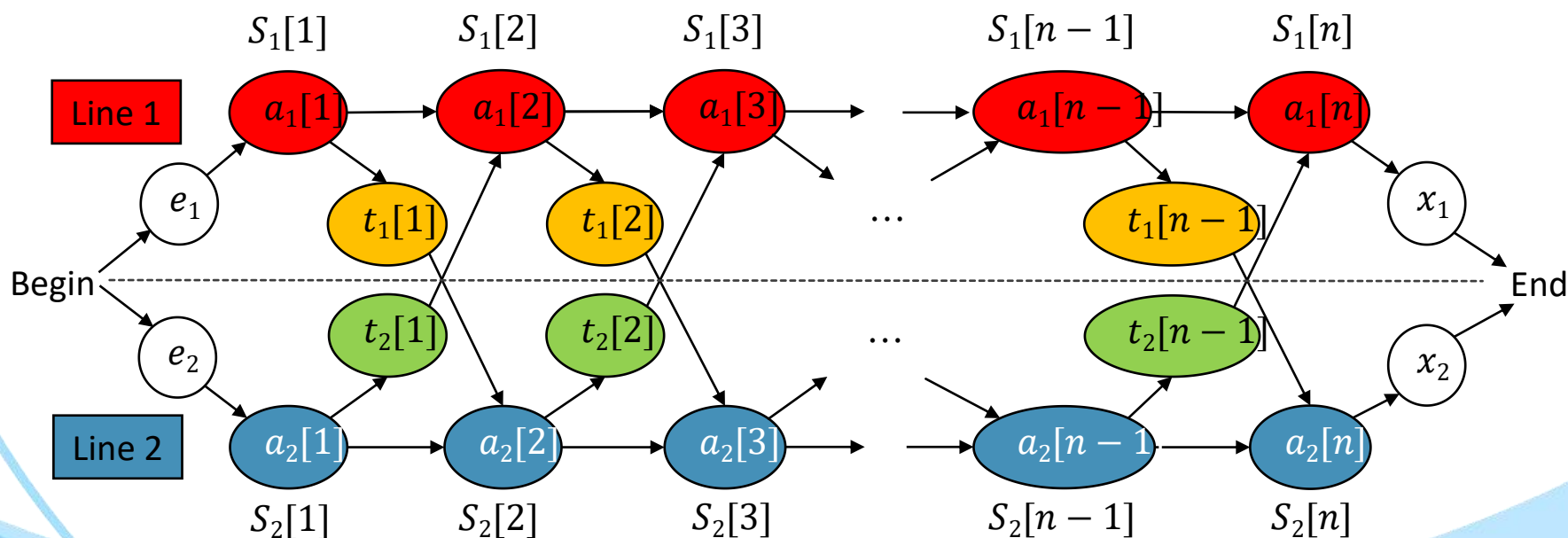
- The j th station on line i ($i = 1$ or 2) is denoted $S_i[j]$ and the **assembly time** at that station is $a_i[j]$.
- At the beginning, a chassis (底盘) takes **e_i time** to enter the factory.
- After going through the j th station on a line, the chassis goes on to the $(j + 1)$ st station on either line.



- There is no transfer cost if it stays on the same line, but it takes **time $t_i[j]$ to transfer** to the other line after station $S_i[j]$.
- After **exiting the n th station** on a line, it **takes x_i time** for the completed auto to exit the factory.
- **The problem** is to determine which stations to choose from line 1 and line 2 in order to **minimize the total time**.

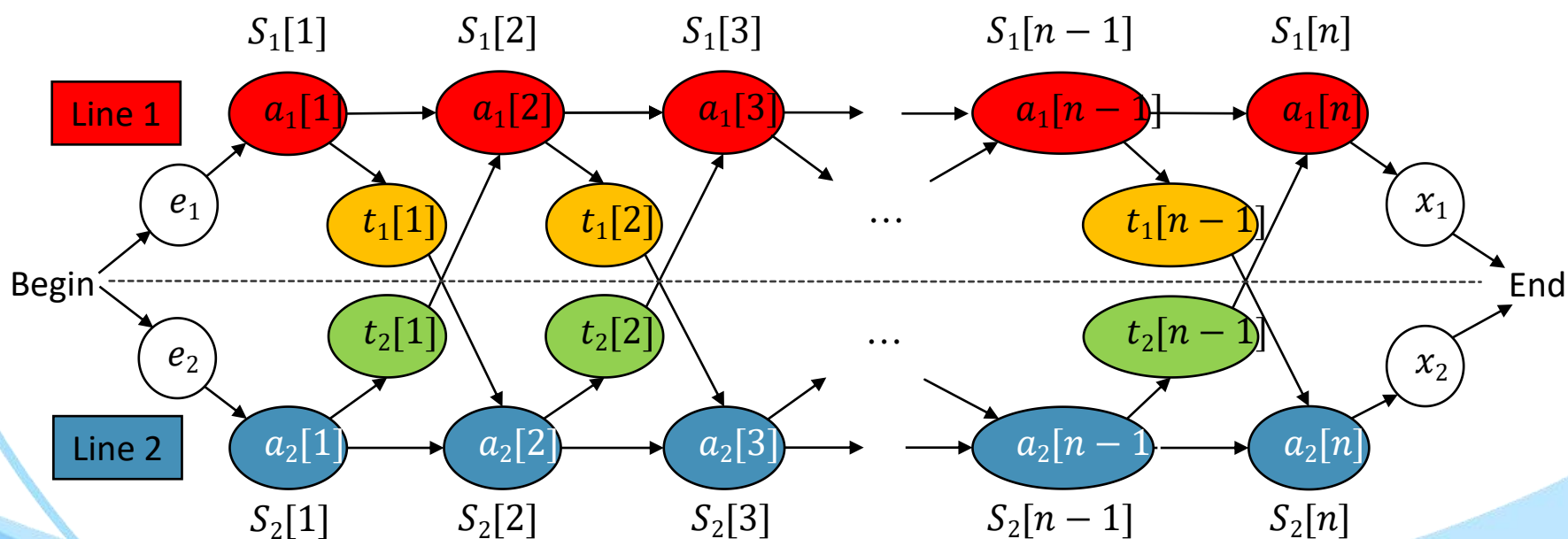


- 一个设备可以在两条流水线之间切换，这个设备需要 n 次的工序才能加工完毕，每次工序 i 不同流水线是不同的时间 $a[i]$ 。如果设备从一条流水线转移到另外一条流水线，需要时间 $t[i]$ 。





- $S_i[j]$: The j th station on line i ($i = 1$ or 2). 第 i 个流水线上第 j 个工序
- $a_i[j]$: The assembly time required at station $S_i[j]$. 加工时间
- $t_i[j]$: The transfer cost from $S_i[j]$ to another line. 传输时间
- e_i : Entry time. 进入某条流水线的时间
- x_i : Exit time. 退出某条流水线的时间

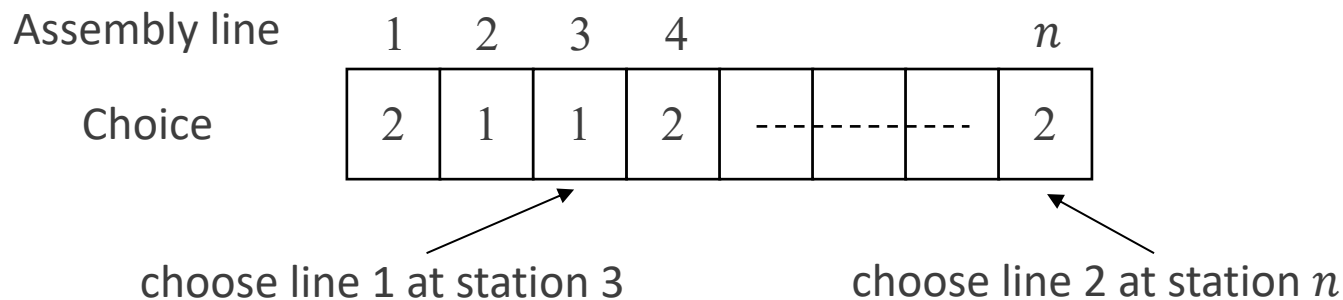




Brute-Force Solution

Brute-force solution (暴力解法):

- **Enumerate** all possibilities of selecting stations. (枚举)
- Compute how long it takes in **each case** and choose the **minimum one**.



- How many possible ways? 2^n
- Is it possible for large n ? **No!**



Structure of an Optimal Solution

- 将流水线装配过程的每个装配点看做不同的调度阶段
- Let's consider **the fastest way** possible to get from the starting point through station $S_1[j]$. 当设备在第一条流水线上j阶段时
 - If $j = 1$: determine how long it takes to get through $S_1[1]$.
 - If $j \geq 2$, have two choices of how to get to $S_1[j]$:
 - Through $S_1[j - 1]$, then directly to $S_1[j]$. 有可能是从j-1阶段来的。
 - Through $S_2[j - 1]$, then transfer over to $S_1[j]$. 可能是从第二条流水线的j-1阶段来的
- Similar for the case through $S_2[j - 1]$.



Structure of an Optimal Solution

- Suppose that the fastest way through station $S_1[j]$ is through station $S_1[j - 1]$. The chassis must have taken a fastest way from the starting point through station $S_1[j - 1]$. 假设到达 $S_1[j]$ 的最优方式必须经过 $s_1[j-1]$ ，那么到达 $s_1[j-1]$ 也是经过最优方式的
 - If from $S_1[j - 1]$ to $S_1[j]$ is optimal, $S_1[j - 1]$ is also optimal.(定理)
- Why?
- Prove by contradiction (反证法来证明) : If there were a faster way to get through station $S_1[j - 1]$, we could substitute this faster way to yield a faster way through station $S_1[j]$: a contradiction. 如果有另外的一个到达 $s_1[j-1]$ 的方式，而且更优，那么我们可以将原有的最优方式的这一部分替代掉，从而得到更好的调度方式，矛盾
 - If $S_1[j - 1]$ is not optimal, from $S_1[j - 1]$ to $S_1[j]$ is also not optimal,



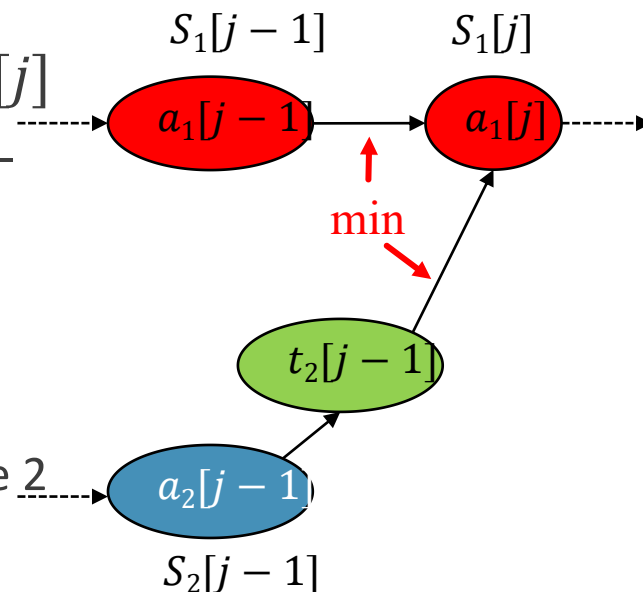
Optimal Substructure

- Generalization: an optimal solution to the problem **find the fastest way through $S_1[j]$** contains within it an optimal solution to subproblems: **find the fastest way through $S_1[j - 1]$ or $S_2[j - 1]$** . 如果一个方法能够找到到达 $S_1[j]$ 的最优路径，那么这个最优路径肯定是能够找到到达 $S_1[j - 1]$ 和 $S_2[j - 1]$ 的最优路径
- This is referred to as the **optimal substructure property** (最优子结构性质).
- 最优子结构：如果我们能找到一个 n 的最优解，那么我们肯定能找到更小规模的问题的最优解
- Optimal substructure property is one of the hallmarks (特征) of the applicability of dynamic programming.
 - We use this property to construct an optimal solution to a problem from optimal solutions to subproblems. (子问题最优解构建原问题最优解)
 - 动态规划如果能够找到最优子结构，那么问题就解决一半了



Optimal Substructure 最优子结构

- Denote $f_i[j]$ as the **fastest time** to get from the starting point through station $S_i[j]$.
- Fastest way through $S_1[j]$** is either: 到达 $S_1[j]$ 的最优路径有两种可能, 一种是经过 $S_1[j-1]$ 一种是经过 $S_2[j-1]$



- fastest way through $S_1[j-1]$ then directly through $S_1[j]$:
$$f_1[j] = f_1[j-1] + a_1[j]$$
- fastest way through $S_2[j-1]$, transfer from line 2 to line 1, then through $S_1[j]$:
$$f_1[j] = f_2[j-1] + t_2[j-1] + a_1[j]$$

- In summary:

$$f_1[j] = \min(f_1[j-1] + a_1[j], f_2[j-1] + t_2[j-1] + a_1[j])$$

Similar for $f_2[j]$

这个式子是非常重要的



Recursive Equation (递归方程)

动态规划的递归方程

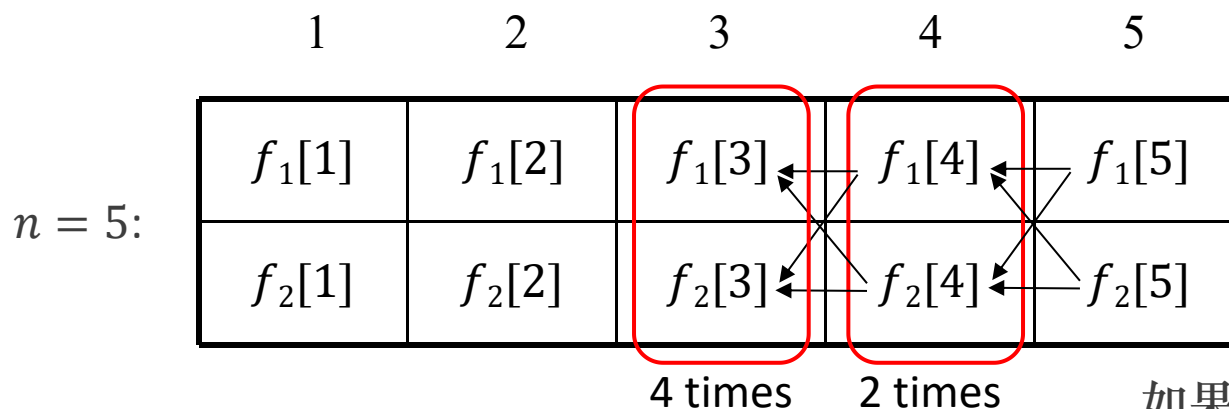
- The recursive equation:

$$f_1[j] = \begin{cases} a_1[1] + e_1 & j = 1 \\ \min(f_1[j-1] + a_1[j], f_2[j-1] + t_2[j-1] + a_1[j]) & j > 1 \end{cases}$$
$$f_2[j] = \begin{cases} a_2[1] + e_2 & j = 1 \\ \min(f_2[j-1] + a_2[j], f_1[j-1] + t_1[j-1] + a_2[j]) & j > 1 \end{cases}$$

- The optimal solution when finishing:

$$f^* = \min \{f_1[n] + x_1, f_2[n] + x_2\}.$$

- How to solve? How about recursion?



Exponential running time!

如果用简单递归方式的话，复杂度是指数级的

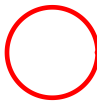
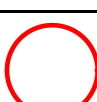


Computing the Optimal Solution

- For $j \geq 2$, each value $f_i[j]$ depends only on the values of: 每个值取决于之前的阶段额两个值

$$f_1[j-1] \text{ and } f_2[j-1].$$

- Compute the values of $f_i[j]$ in **increasing order of j** .

	1	2	3	4	5
$f_1[j]$					
$f_2[j]$					



Calculate column by column in increasing order of j

- Bottom-up** approach: 用自底向上的方法来算
 - First find optimal solutions to subproblems. 实际上就是for循环j从1到n
 - Find an optimal solution to the problem from the subproblems.



Additional Information (求解之后，得到加工信息)

- f^* only records the **optimal cost**. How can we know the decisions at each step? 如何获得在每一个步骤的选择（在哪个流水线加工）
- To construct the **optimal solution**, we need the sequence of what line has been used at each station: 为了知道最优答案，我们要知道每一步用了哪个流水线。
 - $l_i[j]$: the line number (1 or 2) whose previous station ($j - 1$) has been used to get in fastest time through $S_i[j]$, $j = 2, 3, \dots, n$. 在这个阶段的之前一个阶段是在哪条流水线上
 - l^* : the line whose station n is used to get in the fastest way through the entire factory. 最优答案

	2	3	4	5
$l_1[j]$				
$l_2[j]$				


Calculate column by column in increasing order of j



DPFastestWay(a, t, e, x, n)

1 $f_1[1] \leftarrow e_1 + a_1[1]; f_2[1] \leftarrow e_2 + a_2[1]$

2 **for** $j \leftarrow 2$ **to** n **do**

3 **if** $f_1[j-1] \leq f_2[j-1] + t_2[j-1]$ **then**

4 $f_1[j] \leftarrow f_1[j-1] + a_1[j]$

5 $l_1[j] \leftarrow 1$

6 **else** $f_1[j] \leftarrow f_2[j-1] + t_2[j-1] + a_1[j]$

7 $l_1[j] \leftarrow 2$

8 **if** $f_2[j-1] \leq f_1[j-1] + t_1[j-1]$ **then**

9 $f_2[j] \leftarrow f_2[j-1] + a_2[j]$

10 $l_2[j] \leftarrow 2$

11 **else** $f_2[j] \leftarrow f_1[j-1] + t_1[j-1] + a_2[j]$

12 $l_2[j] \leftarrow 1$

13 **if** $f_1[n] + x_1 \leq f_2[n] + x_2$ **then**

14 $f^* \leftarrow f_1[n] + x_1$

15 $l^* \leftarrow 1$

16 **else** $f^* \leftarrow f_2[n] + x_2$

17 $l^* \leftarrow 2$

From line 1 to **line 1**

From line 2 to **line 1**

From line 2 to **line 2**

From line 1 to **line 2**

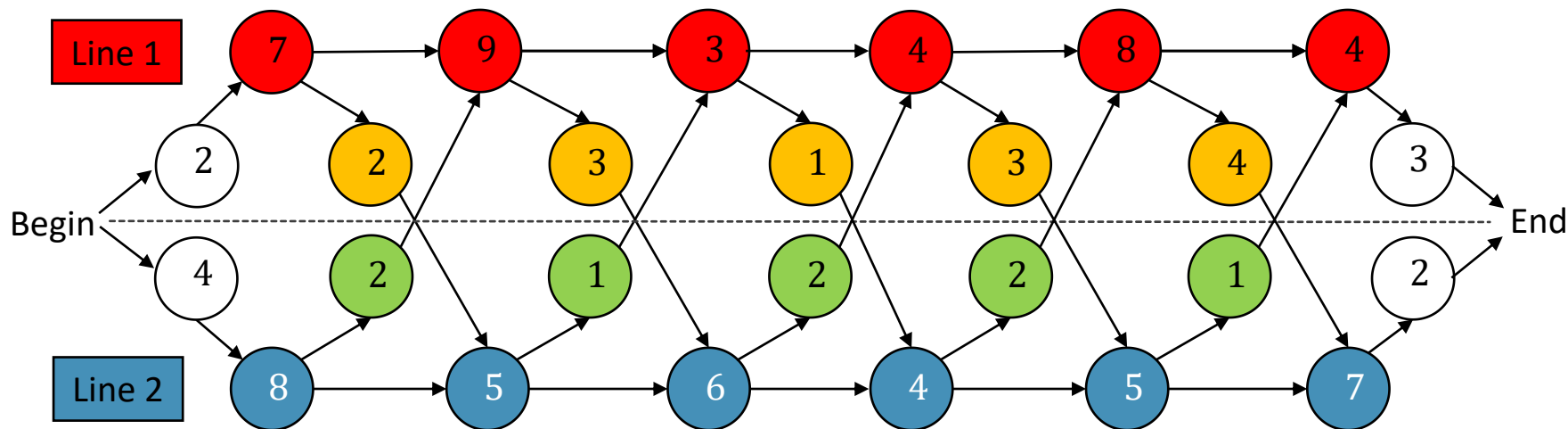
$S_1[j]$

$S_2[j]$

Running time: $\Theta(n)$



Example (举例说明)



j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

$$f^* = 38$$

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$$l^* = 1$$



Construct an Optimal Solution （构造最优答案）

- After calculating, how can we know the optimal solution?

PrintStations(l, n)

```
1  $i \leftarrow l^*$ 
2 print “line ”  $i$  “, station ”  $n$ 
3 for  $j \leftarrow n$  downto 2 do
4      $i \leftarrow l_i[j]$ 
5     print “line ”  $i$  “, station ”  $j - 1$ 
```

Output:

line 1, station 6

line 2, station 5

line 2, station 4

line 1, station 3

line 2, station 2

line 1, station 1

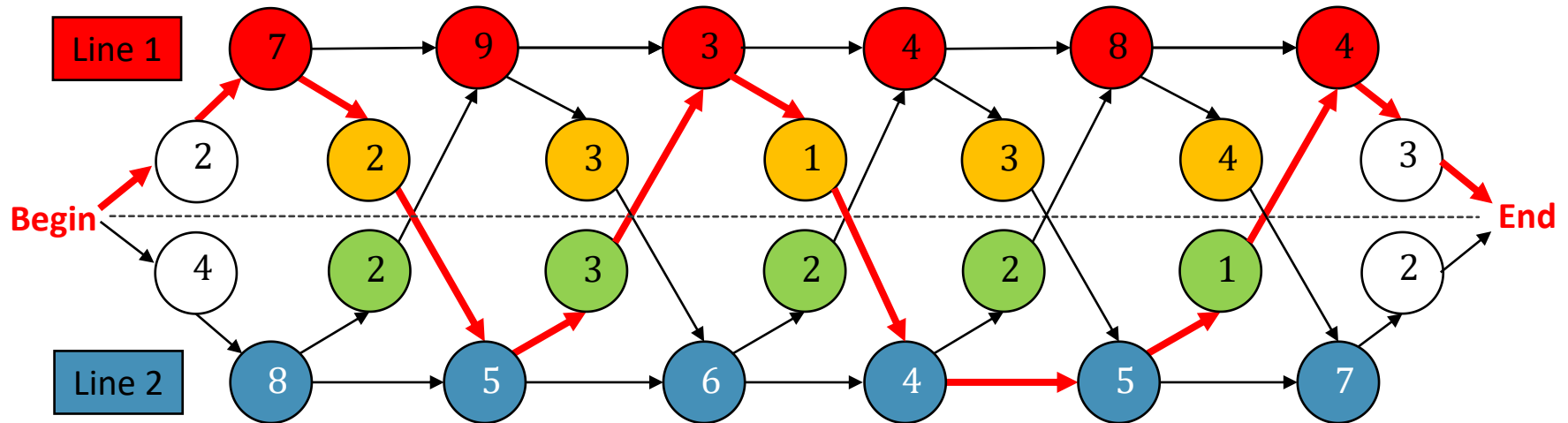
j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$l^* = 1$



Construct an Optimal Solution

The fastest assembly way: $f^* = 38$





Optimal Solutions

- Dynamic programming **not only** solves **the optimal solution** for $n = 6$, but for **all n** ($n \leq 6$).
 - Each cell in the table records **the optimal solution**.
- Every optimal solution is built upon **previous** optimal solutions. Thus, everything in the table is an optimal solution! 最优结果依赖于之前步骤的最优结果

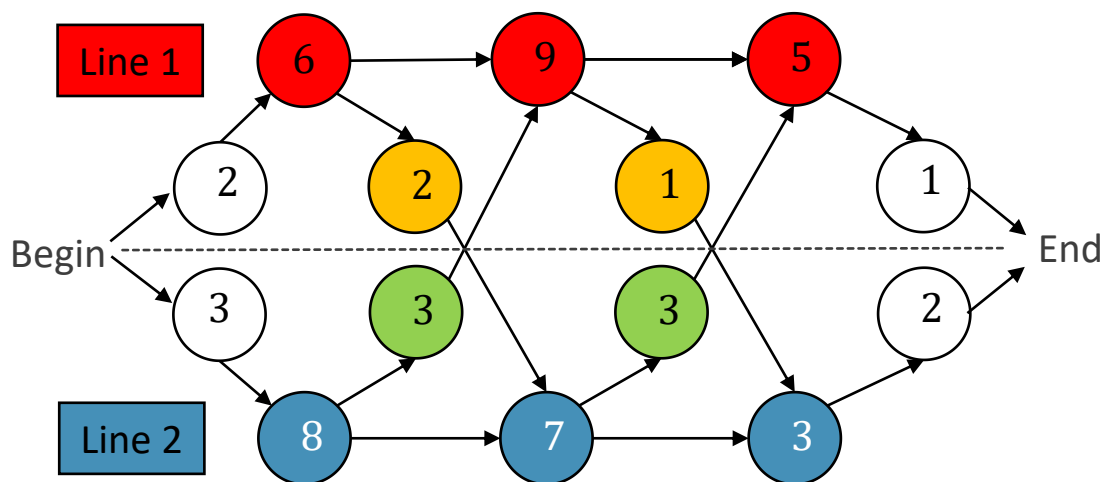
j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2



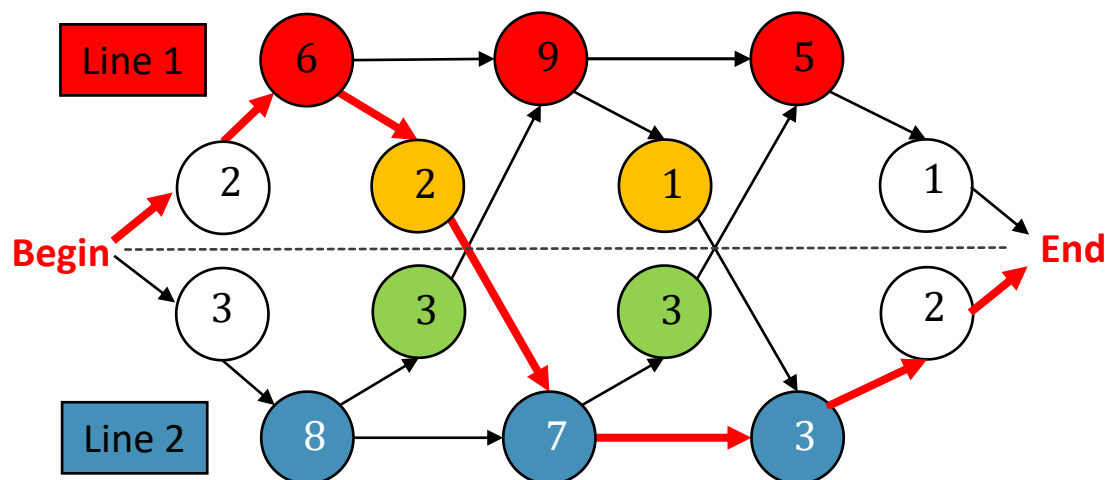
Classroom Exercise (课堂练习)

Write the table of $f[j]$ and $l[j]$ for the following problem and determine the optimal solution:





Classroom Exercise (课堂练习)



j	1	2	3
$f_1[j]$	8	17	22
$f_2[j]$	11	17	20

$$f^* = 22$$

j	2	3
$l_1[j]$	1	1
$l_2[j]$	1	2

$$l^* = 2$$



MATRIX-CHAIN MULTIPLICATION

矩阵乘法



Matrix-Chain Multiplication

Matrix-chain multiplication (矩阵链乘法) problem:

- Given a chain $A_1, A_2, \dots, A_n$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$.

$$\begin{array}{ccccccc} A_1 & \cdot & A_2 & \cdots & A_i & \cdot & A_{i+1} & \cdots & A_n \\ p_0 \times p_1 & p_1 \times p_2 & p_{i-1} \times p_i & p_i \times p_{i+1} & p_{n-1} \times p_n \end{array}$$

- Matrix production satisfies the associative law (结合律). Thus, the order of calculation doesn't influence the production result, but influence the efficiency.
- 矩阵乘法满足结合律，不同顺序的结果不改变结果，但是有不同的运算量
- 目标：通过加括号来改变计算顺序的方式，从而降低矩阵连乘的运算量
- Goal:** fully parenthesize (加括号) the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications.



Matrix Multiplication 复习下矩阵乘

- To multiply an $n \times m$ matrix with a $m \times q$ matrix using the standard method, it is necessary to do $n \times m \times q$ elementary multiplications.

```
MatrixMultiply(A, B)
1  if  $m \neq p$  then
2      print "Two matrices cannot multiply"
3  else for  $i \leftarrow 1$  to  $n$ 
4      for  $j \leftarrow 1$  to  $q$  do
5           $C[i, j] \leftarrow 0$ 
6          for  $k \leftarrow 1$  to  $m$  do
7               $C[i, j] \leftarrow C[i, j] + A[i, k]B[k, j]$ 
8  return  $C$ 
```

Running time:
 $\Theta(nmq)$



Example (例子)

- Consider the chained matrix multiplication:

$$\begin{array}{ccccccc} A & \times & B & \times & C & \times & D \\ 20 \times 2 & & 2 \times 30 & & 30 \times 12 & & 12 \times 8 \end{array}$$

- The total number of elementary multiplications depends on the multiplication order.

$$A(B(CD)): 30 \times 12 \times 8 + 2 \times 30 \times 8 + 20 \times 2 \times 8 = 3,680$$

$$(AB)(CD): 20 \times 2 \times 30 + 30 \times 12 \times 8 + 20 \times 30 \times 8 = 8,880$$

$$A((BC)D): 2 \times 30 \times 12 + 2 \times 12 \times 8 + 20 \times 2 \times 8 = 1,232$$

$$((AB)C)D: 20 \times 2 \times 30 + 20 \times 30 \times 12 + 20 \times 12 \times 8 = 10,320$$

$$(A(BC)D): 2 \times 30 \times 12 + 20 \times 2 \times 12 + 20 \times 12 \times 8 = 3,120$$



Counting the Number of Parenthesizations

- Brute-force solution: Find them all and pick the smallest!
- How many ways can we parenthesize the product of a matrix chain? 我们到底有多少种加括号的方式（来改变运算方式）
 - Denote $P(n)$: The number of alternative parenthesizations of a sequence of n matrices.
 - We can split a sequence of n matrices between the k th and $(k + 1)$ st matrices for any $k = 1, 2, \dots, n - 1$ and then parenthesize the two resulting subsequences independently.

$$(A_1 A_2 \dots A_k)(A_{k+1} A_{k+2} \dots A_n)$$

- Sum over all $k = 1, 2, \dots, n - 1$ with recursive calculation.



Counting the Number of Parenthesizations

- $P(n)$ 到底有几种加括号的方式,
- We obtain the recursive equation:

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n \geq 2 \end{cases}$$

- The solution to the above recursive equation is $\Omega(2^n)$.



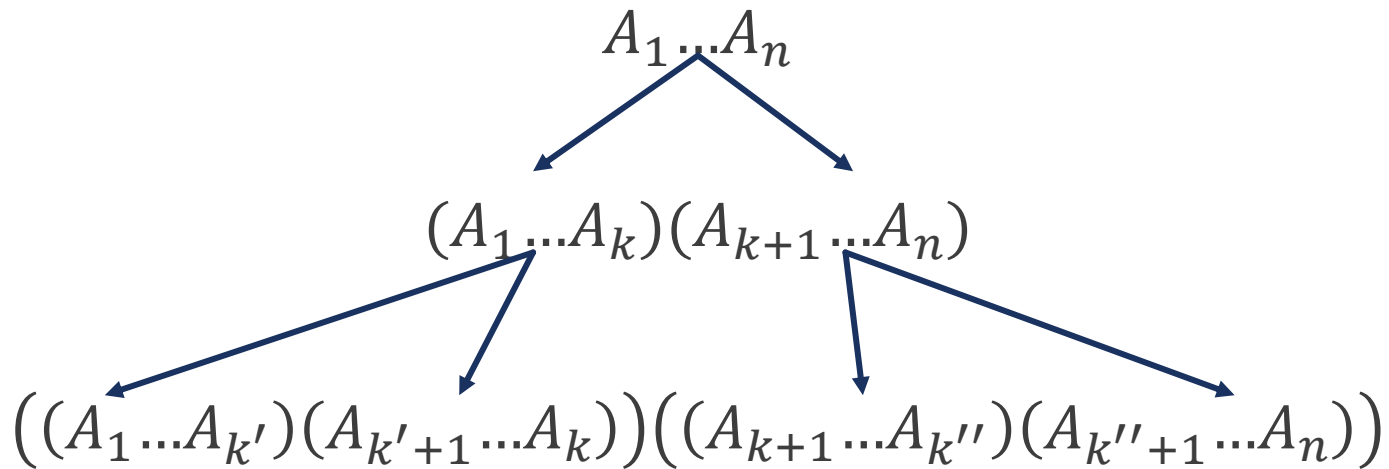
Optimal Substructure

动态规划 最优子结构

- Assume we have found an optimal solution for parenthesizing $A_1A_2\dots A_n$.
- The optimal parenthesization must be with some k , for $1 \leq k \leq n - 1$: **假设最优**的加括号的方式是加到 k 的。
$$(A_1 \dots A_k)(A_{k+1} \dots A_n)$$
- Then, the parenthesization for $A_1 \dots A_k$ and $A_{k+1} \dots A_n$ **must also be optimal. 最优子结构**: $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 的加括号的方式**也是最优的**
 - Why? Prove by contradiction again!
- We find the optimal substructure: **An optimal solution to an instance of the matrix-chain multiplication is constructed by the optimal solutions to subproblems.**(矩阵链乘法实例的最优解由子问题的最优解构造)



Optimal Substructure



- Therefore, we need to consider the optimal solution for $A_i A_{i+1} \dots A_j$, for arbitrary $1 \leq i \leq j \leq n$.



Recursive Equation 递归方程

- We define $A_{i...j} = A_i A_{i+1} \cdots A_j$ and let $m[i, j]$ = **the minimum number** of multiplications needed to compute $A_{i...j}$. $m[i, j]$ 来算从*i*到*j*所需要的最小计算量

- **Suppose that an optimal** parenthesization of $A_{i...j}$ splits the product **between A_k and A_{k+1}** , where $i \leq k < j$:

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

Optimal cost to
calculate $A_{i...k}$

Optimal cost to
calculate $A_{k+1...j}$

Cost to calculate
 $(A_{i...k})(A_{k+1...j})$

- Now, if we have known all the optimal costs to the small instances, how to construct the optimal cost to the current instance?

Iterate over all k for $i \leq k < j$ and select the minimum one!

遍历所有的k，选择使得m[i,j] 最小的那个



Recursive Equation

- There are $j - i$ possible values for k : $k = i, i + 1, \dots, j - 1$.
- Minimizing the cost of parenthesizing $A_{i\dots j}$ becomes:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j \} & i < j \end{cases}$$

- The original problem is solved by calculating $m[1, n]$.
- How to calculate?

如果直接用递归的话，由于大量重叠的子问题，会导致大量浪费的重复计算，因此我们在规划里面，用自底向上的方式求解上面的递归方程（所有的递归都可以写成递推的方式）



Filling Table

- To avoid repeated computation, we use a **table to store** computed values of $m[i, j]$.

- Given the recursive equation

$$m[i, j] = \min_{i \leq k < j} m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$$

what components do we need to calculate $m[i, j]$?

- Fix i , all $m[i, k]$ for $i \leq k < j$.
- Fix j , all $m[k + 1, j]$ for $i \leq k < j$.



Filling Table

- Calculating $m[i, j]$ only requires:
 - $m[k + 1, j]$ for $i \leq k < j$: The columns behind $m[i, j]$ on the same row.
 - $m[i, k]$ for $i \leq k < j$: The rows below $m[i, j]$ on the same column.

i

	1	2	3	4	...	n
n						
:		$m[i, j]$	$m[i + 1, j]$	...	$m[j, j]$	
4		$m[i, j + 1]$				
3		...				
2		$m[i, i]$				
1						

注意纵坐标的顺序 j

What are the cells in the shaded area?

灰色的是无效的
计算，
要求 $i \leq j$



Example

- Given the recursive equation:

$$\min_{i \leq k < j} m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$$

- Calculating $m[2,5]$:

$$\min \begin{cases} m[2,2] + m[3,5] + p_1p_2p_5 & k = 2 \\ m[2,3] + m[4,5] + p_1p_3p_5 & k = 3 \\ m[2,4] + m[5,5] + p_1p_4p_5 & k = 4 \end{cases}$$

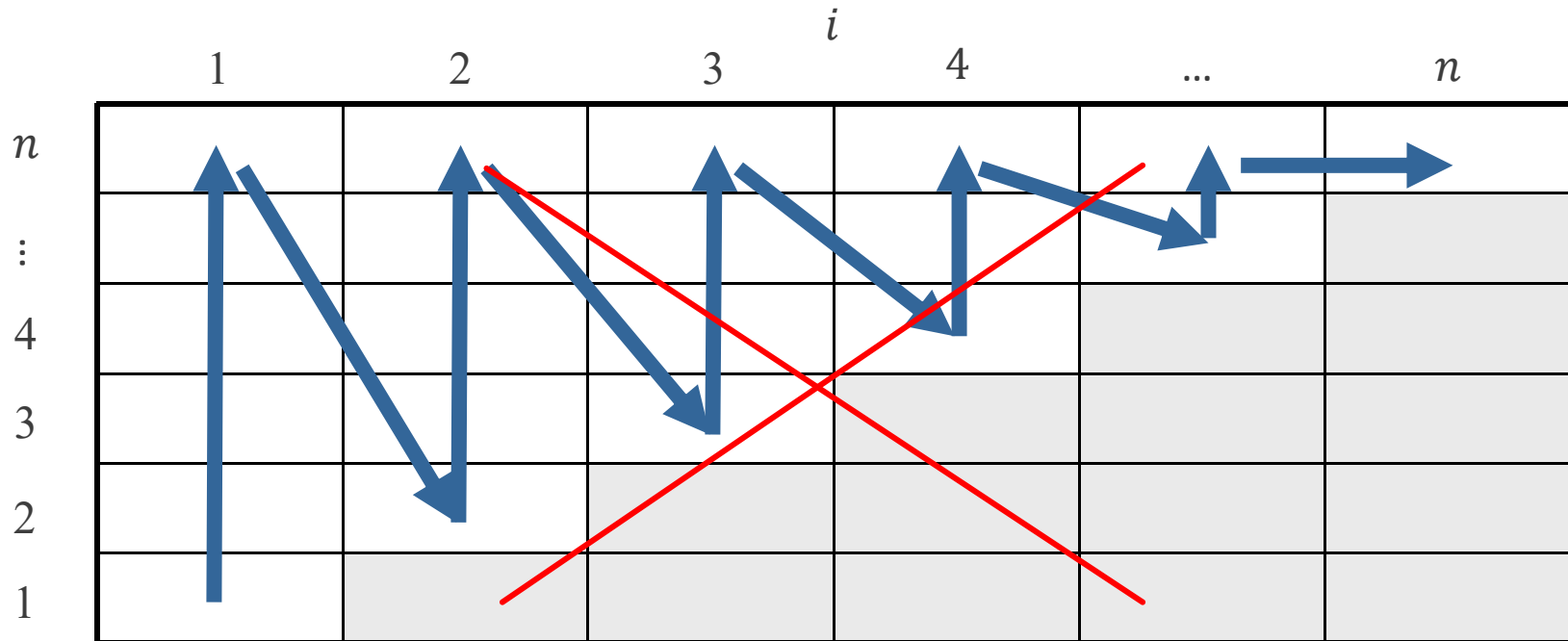
注意纵坐标的顺序

		i					
		1	2	3	4	5	n
j	n						
	5						
	4						
	3						
	2						
	1						



Filling Table (自底向上填表的过程)

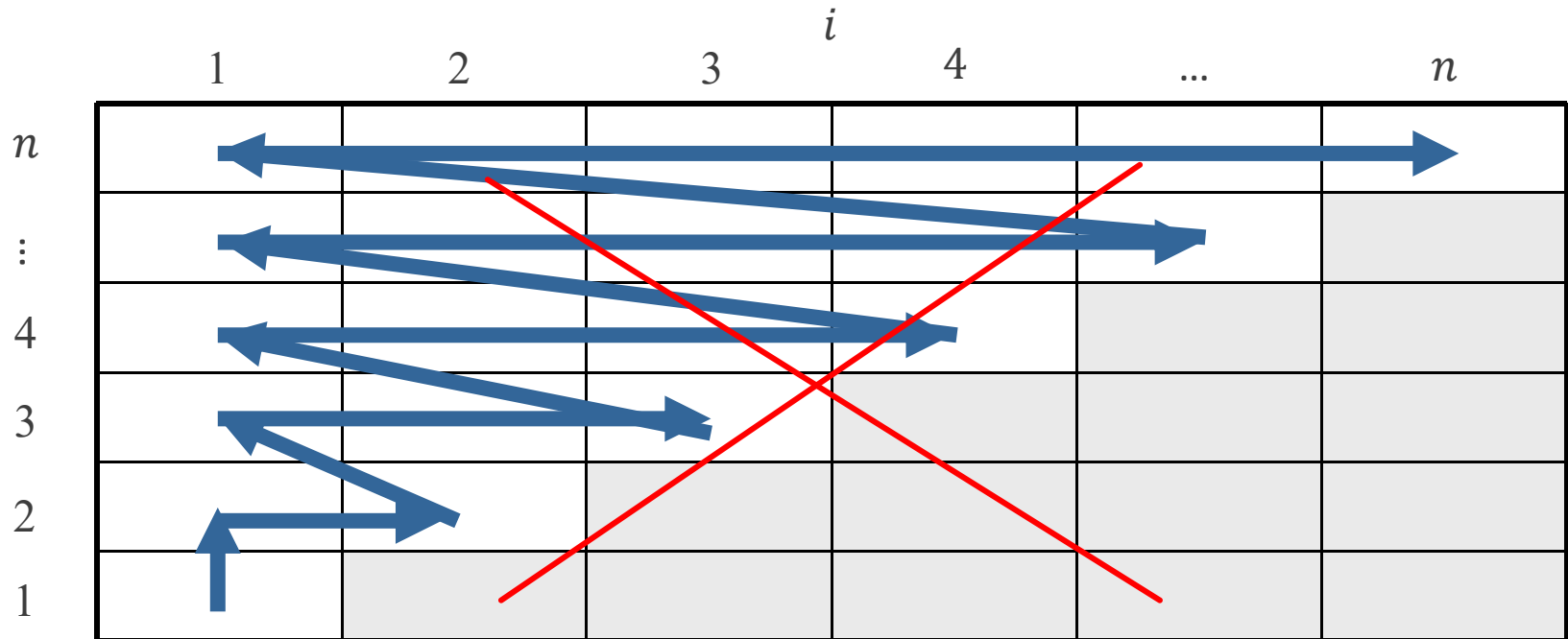
■ Can we filling the table in this order?





Filling Table (自底向上填表的过程)

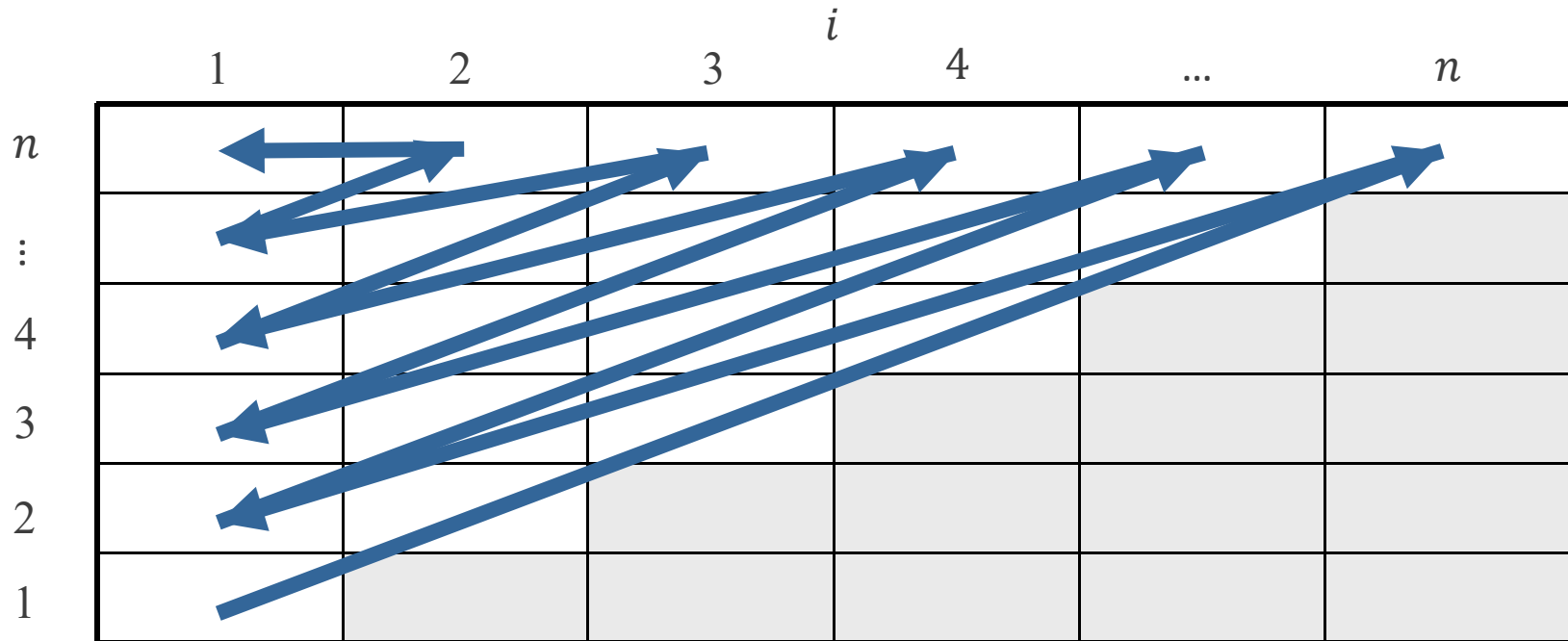
■ How about this order?





Filling Table (自底向上填表的过程)

- Filling the table **diagonal by diagonal**. 对角线的方式





Example (举例)

■ Given:

- $A_1: 10 \times 100$ ($p_0 \times p_1$)
- $A_2: 100 \times 5$ ($p_1 \times p_2$)
- $A_3: 5 \times 50$ ($p_2 \times p_3$)

■ Calculate:

- $m[i, i] = 0$ for $i = 1, 2, 3$.
- $(A_1A_2): m[1, 2] = m[1,1] + m[2,2] + p_0p_1p_2 = 0 + 0 + 10 \times 100 \times 5 = 5,000$.
- $(A_2A_3): m[2, 3] = m[2,2] + m[3,3] + p_1p_2p_3 = 0 + 0 + 100 \times 5 \times 50 = 25,000$.
- $(A_1(A_2A_3)): m[1,1] + m[2,3] + p_0p_1p_3 = 75,000$.
- $((A_1A_2)A_3): m[1,2] + m[3,3] + p_0p_2p_3 = 7,500$.

		i		
		1	2	3
j	3	7500	25000	0
	2	5000	0	
	1	0		



Reconstructing the Optimal Solution

- 如何得到最优解的过程
- Again, we need additional information to **maintain the optimal solution** to the optimal cost.
- 用 $s[i, j]$ 来记录要在 $A_{i...j}$ 的第 k 个位置切分
- Let $s[i, j]$ = **a value of k** at which we can split the product $A_{i...j}$ in order to obtain an optimal parenthesization.



Pseudocode

DPMatrixChain(p)

```
1  for  $i \leftarrow 1$  to  $n$  do First diagonal
2       $m[i, i] \leftarrow 0$ 
3  for  $c \leftarrow 2$  to  $n$  do From 2nd to  $n$ th diagonal
4      for  $i \leftarrow 1$  to  $n - c + 1$  do
5           $j \leftarrow i + c - 1$  Given  $c$ , determine the  $j$ th column and  $i$ th row
6           $m[i, j] \leftarrow \infty$ 
7          for  $k \leftarrow i$  to  $j - 1$  do
8               $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
9              if  $q < m[i, j]$  then
10                  $m[i, j] \leftarrow q; s[i, j] \leftarrow k$ 
11 return  $m$  and  $s$ 
```

Running time: $\Theta(n^3)$



Construct the Optimal Solution

- $s[i, j]$ stores the value of k such that the optimal solution of $A_{i...j}$ splits the product into $A_{i...k}$ and $A_{k+1...j}$.

		1	2	3	i 4	5	6
6		3	3	3	5	5	-
5		3	3	3	4	-	
4		3	3	3	-		
j 3		1	2	-			
2		1	-				
1		-					

$$A_{1...n} = A_{1...s[1,n]} \cdot A_{s[1,n]+1...n}$$

$$A_{1...6} = A_{1...3} A_{4...6}$$

$$A_{1...3} = A_{1...1} A_{2...3}$$

$$A_{4...6} = A_{4...5} A_{6...6}$$



PrintParens(s, i, j)

1 if $i = j$ then print " A_i "

2 else print "("

3 PrintParens($s, i, s[i, j]$)

4 PrintParens($s, s[i, j] + 1, j$)

5 print ")"

		i					
		1	2	3	4	5	6
6		3	3	3	5	5	-
5		3	3	3	4	-	
4		3	3	3	-		
j 3		1	2	-			
2		1	-				
1		-					

Function call	Printed value
PrintParens($s, 1, 6$)	(
PrintParens($s, 1, 3$)	((
PrintParens($s, 1, 1$)	((A_1
PrintParens($s, 2, 3$)	((A_1 (
PrintParens($s, 2, 2$)	((A_1 (A_2
PrintParens($s, 3, 3$)	((A_1 (A_2 A_3
PrintParens($s, 4, 6$)	((A_1 (A_2 A_3))(
PrintParens($s, 4, 5$)	((A_1 (A_2 A_3)))((
PrintParens($s, 4, 4$)	((A_1 (A_2 A_3)))((A_4
PrintParens($s, 5, 5$)	((A_1 (A_2 A_3)))((A_4 A_5
PrintParens($s, 6, 6$)	((A_1 (A_2 A_3)))((A_4 A_5) A_6
Finish	((A_1 (A_2 A_3)))((A_4 A_5) A_6))



Classroom Exercise (课堂练习)

- Given the following matrices, fill in the table to get the optimal parenthesization: 算出最佳乘法，要填m矩阵和s矩阵
 - $A_1: 2 \times 2$
 - $A_2: 2 \times 4$
 - $A_3: 4 \times 2$
 - $A_4: 2 \times 6$



Classroom Exercise

$$A_1: 2 \times 2$$

$$A_2: 2 \times 4$$

$$A_3: 4 \times 2$$

$$A_4: 2 \times 6$$

$$m[i, j] = \min_{i \leq k < j} m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

		<i>i</i>			
		1	2	3	4
<i>j</i>	4				0
	3			0	
	2		0		
	1	0			

$m[i, j]$

		<i>i</i>			
		1	2	3	4
<i>j</i>	4				-
	3			-	
	2		-		
	1	-			

$s[i, j]$



Classroom Exercise

$$A_1: 2 \times 2$$

$$A_2: 2 \times 4$$

$$A_3: 4 \times 2$$

$$A_4: 2 \times 6$$

$$m[i, j] = \min_{i \leq k < j} m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

		<i>i</i>			
		1	2	3	4
<i>j</i>	4	48	40	48	0
	3	24	16	0	
	2	16	0		
	1	0			

$m[i, j]$

		<i>i</i>			
		1	2	3	4
<i>j</i>	4	3	3	3	-
	3	1	2	-	
	2	1	-		
	1	-			

$s[i, j]$

$$((A_1(A_2A_3))A_4)$$



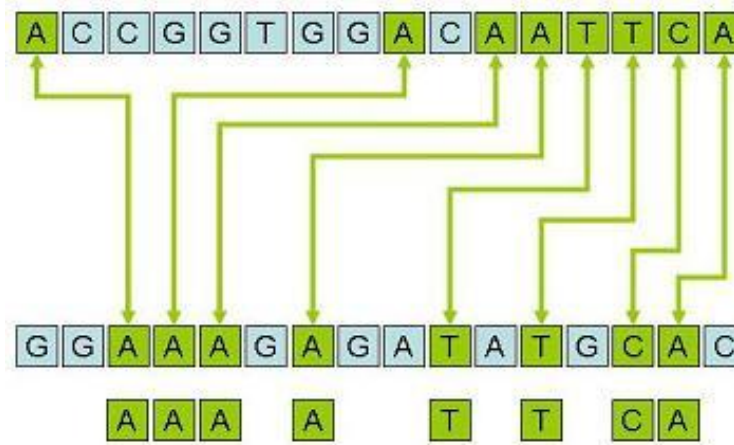
THE LONGEST-COMMON-SUBSEQUENCE PROBLEM

最长公共子序列问题



The Longest-Common-Subsequence Problem

- In biological applications, one goal of comparing two strands of DNA is to determine how "similar" the two strands are, as some measure of how closely related the two organisms are.
- 在生物应用中，经常通过比较两个DNA链的方式来比较两者是不是比较像。用字符串来表示DNA链，找两个字符串中最长的公共子串
- 子串的元素之间可以不连续，但是必须是按照从左到右的顺序
- One way to measure the similarity between S_1 and S_2 is by finding a third strand S_3 : the bases in S_3 appear in each of S_1 and S_2 ; these bases must appear in the same order, but not necessarily consecutively.





The Longest-Common-Subsequence Problem

- Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$, another sequence $Z = \langle z_1, z_2, \dots, z_k \rangle$ is a **subsequence (子序列)** of X if there exists a **strictly increasing sequence** $i_1, i_2, \dots, i_k$ of indices of X such that for all $j = 1, 2, \dots, k$, we have $x_{i_j} = z_j$.

如果说 z 是 x 的子序列，那么 z 的所有元素都在 x 中，并且是在 x 中是从左到右（不一定相邻）排列的

- Given two sequences X and Y , we say that a sequence Z is a **common subsequence (公共子序列)** of X and Y if Z is a subsequence of both X and Y . 如果一个 z 同时是 x 和 y 的子序列，那么 z 是他们的公共子序列
- In the **longest-common-subsequence (最长公共子序列)** problem, we are given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ and wish to find a maximum-length common subsequence of X and Y . 最长公共子序列就是公共子序列里面最长



Example

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

- $\langle B, C, B, A \rangle$ and $\langle B, D, A, B \rangle$ are longest common subsequences of X and Y ($length = 4$).
- $\langle B, C, A \rangle$ is a common subsequence, but not the longest.



Brute-Force Solution 暴力解法

$$X_m = \langle x_1, x_2, \dots, x_m \rangle$$

$$Y_n = \langle y_1, y_2, \dots, y_n \rangle$$

- For every subsequence of X , check whether it's a subsequence of Y . 对于每个 x 的子序列，检查是不是 y 的子序列
- There are 2^m subsequences of X to check.
- Each subsequence takes $\Theta(n)$ time to check.
 - Scan Y from the first element, and check if it is matched with the subsequence.
- Running time: $\Theta(n2^m)$.



Optimal Substructure 最优子结构

- Given two sequences $X_m = \langle x_1, x_2, \dots, x_m \rangle$ and $Y_n = \langle y_1, y_2, \dots, y_n \rangle$, assume the $Z_k = \langle z_1, z_2, \dots, z_k \rangle$ is a **LCS**.

给定 X_m 和 Y_n 的最长公共子序列 Z_k

- If the last element of X_m and Y_n **is same**, i.e. $x_m = y_n$, they are **also same** as the last element of Z_k : 如果X和Y的最后一个元素一样

$$Z_k = x_m = y_n = \blacksquare$$

$$X_m = \langle X_{m-1}, \blacksquare \rangle, Y_n = \langle Y_{n-1}, \blacksquare \rangle, Z_k = \langle Z_{k-1}, \blacksquare \rangle$$

- 那么Z的最后一个元素也是和X和Y的最后一个元素一样，并且 Z_{k-1} 也是 X_{m-1} 和 Y_{n-1} 的最长公共子序列
- In this case, Z_{k-1} is **also a LCS** of X_{m-1} and Y_{n-1} .
 - Prove by contradiction. 通过反证法来证明这个性质



Optimal Substructure 最优子结构

- Given two sequences $X_m = \langle x_1, x_2, \dots, x_m \rangle$ and $Y_n = \langle y_1, y_2, \dots, y_n \rangle$, assume the $Z_k = \langle z_1, z_2, \dots, z_k \rangle$ is a **LCS**.
- If the last element of X_m and Y_n is different, i.e. $x_m \neq y_n$:
- 如果X和Y的最后一个元素不一样
 - If $z_k \neq x_m$, 如果 $z_k \neq x_m$ 那么 **Z_k 是 LCS of X_{m-1} and Y_n .**
 $X_m = \langle X_{m-1}, \blacksquare \rangle, Y_n, Z_k = \langle Z_{k-1}, \blacklozenge \rangle$
no matter $z_k = y_n$ or not, Z_k is a **LCS** of X_{m-1} and Y_n .
 - If $z_k \neq y_n$, 那么 **Z_k 是 LCS of X_m and Y_{n-1} .**
 $X_m, Y_n = \langle Y_{n-1}, \blacksquare \rangle, Z_k = \langle Z_{k-1}, \blacklozenge \rangle$
no matter $z_k = x_m$ or not, **Z_k is a LCS of X_m and Y_{n-1} .**



Optimal Substructure

- Denote $c[i, j]$ as the length of a LCS of the sequences $X_i = \langle x_1, x_2, \dots, x_i \rangle$ and $Y_j = \langle y_1, y_2, \dots, y_j \rangle$. $c[i, j]$ 是 $X_i Y_j$ 最长子序列的长度
- If $x_i = y_j$, the LCS composes of the LCS of X_{i-1} and Y_{j-1} with x_i :

$$c[i, j] = c[i - 1, j - 1] + 1.$$

- If $x_i \neq y_j$, the LCS is either the LCS of X_{i-1} and Y_j , or the LCS of X_i and Y_{j-1} . We choose the longer one:

$$c[i, j] = \max \{c[i - 1, j], c[i, j - 1]\}.$$



Example

- Case 1: $x_i = y_j$

$$X_i = \langle A, B, D, E \rangle, \quad Y_j = \langle Z, B, E \rangle, \quad Z_k = \langle B, E \rangle$$

$$X_{i-1} = \langle A, B, D \rangle, \quad Y_{j-1} = \langle Z, B \rangle, \quad Z_{k-1} = \langle B \rangle$$

- Case 2: $x_i \neq y_j$

$$X_i = \langle A, B, D, G \rangle, \quad Y_j = \langle Z, B, D \rangle, \quad Z_k = \langle B, D \rangle$$

Z_k is **either** the LCS of X_{i-1} and Y_j **or** the LCS of X_i and Y_{j-1} .



Recursive Equation

- The recursive equation:

$$c[i, j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i - 1, j - 1] + 1 & i, j > 0 \text{ and } x_i = y_j \\ \max \{c[i - 1, j], c[i, j - 1]\} & i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

- In the case of $i = 0$ or $j = 0$, the length of LCS is 0 because it is empty.
- Using recursion, is there any repeated computation?



Filling Table

$$c[i, j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i - 1, j - 1] + 1 & i, j > 0 \text{ and } x_i = y_j \\ \max \{c[i - 1, j], c[i, j - 1]\} & i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

	0	1	2	j	...	n
0	0	0	0	0	0	0
1	0					
2	0					
i	0					
$\vdots$	0					
m	0					

Diagram illustrating the filling of the DP table. The table is a grid with rows indexed by i and columns indexed by j . The first row and first column are filled with 0. The rest of the table is filled with values. Blue arrows show the flow of computation from top-left to bottom-right. Red arrows highlight the recurrence relation for $c[i, j]$, showing it depends on $c[i-1, j-1]$, $c[i-1, j]$, and $c[i, j-1]$.



Additional Information

- $c[i, j]$ only records the length of LCS, we need to **record additional information for constructing** LCS.
- $b[i, j]$ records the choices made to obtain the optimal value. 记录从哪里来的

```
if  $x_i = y_j$ 
     $b[i, j] = \text{"↖"}$ 
else if  $c[i - 1, j] \geq c[i, j - 1]$ 
     $b[i, j] = \text{"↑"}$ 
else
     $b[i, j] = \text{"←"}$ 
```

		j					
		0	1	2	...		n
i	0	0	0	0	0	0	0
	1	0					
	2	0					
	$\vdots$	0					
	$\vdots$	0					
	m	0					



Pseudocode

```
LCSLength( $X, Y, m, n$ )
1  for  $i \leftarrow 1$  to  $m$  do  $c[i, 0] \leftarrow 0$ 
2  for  $j \leftarrow 0$  to  $n$  do  $c[0, j] \leftarrow 0$ 
3  for  $i \leftarrow 1$  to  $m$  do
4      for  $j \leftarrow 1$  to  $n$  do
5          if  $x_i = y_j$  then
6               $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ 
7               $b[i, j] \leftarrow \nwarrow$ 
8          else if  $c[i - 1, j] \geq c[i, j - 1]$  then
9               $c[i, j] \leftarrow c[i - 1, j]$ 
10              $b[i, j] \leftarrow \uparrow$ 
11          else  $c[i, j] \leftarrow c[i, j - 1]$ 
12              $b[i, j] \leftarrow \leftarrow$ 
13  return  $c$  and  $b$ 
```

Running time:
 $\Theta(nm)$



Example

if $x_i = y_j$ then

$c[i, j] \leftarrow c[i - 1, j - 1] + 1$

$b[i, j] \leftarrow "\nwarrow"$

else if $c[i - 1, j] \geq c[i, j - 1]$ then

$c[i, j] \leftarrow c[i - 1, j]$

$b[i, j] \leftarrow "\uparrow"$

else $c[i, j] \leftarrow c[i, j - 1]$

$b[i, j] \leftarrow "\leftarrow"$

$X = \langle B, D, C, A, B, A \rangle$

$Y = \langle A, B, C, B, D, A \rangle$

		i						
		0	1	2	3	4	5	6
		x_i	B	D	C	A	B	A
i	0 y_j	0	0	0	0	0	0	0
	1 A	0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\swarrow$ 1	$\leftarrow$ 1	$\swarrow$ 1
	2 B	0	$\swarrow$ 1	$\leftarrow$ 1	$\leftarrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2
	3 C	0	$\uparrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2	$\uparrow$ 2	$\uparrow$ 2
	4 B	0	$\swarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\leftarrow$ 3
	5 D	0	$\uparrow$ 1	$\swarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\uparrow$ 3
	6 A	0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\uparrow$ 3	$\swarrow$ 4
	7 B	0	$\swarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\swarrow$ 4	$\uparrow$ 4



Example

- Constructing a LCS: **Start** at $b[m, n]$ and **follow the arrows**. When we encounter a " $\nwarrow$ " in $b[i, j]$, it means $x_i = y_j$ and it is an **element** of the LCS.

PrintLCS(b, X, i, j)

```

1 if  $i = 0$  or  $j = 0$  then return 0
2 if  $b[i, j] = "\nwarrow"$  then
3   PrintLCS( $b, X, i - 1, j - 1$ )
4   print  $x_i$ 
5 else if  $b[i, j] = "\uparrow"$  then
6   PrintLCS( $b, X, i - 1, j$ )
7 else PrintLCS( $b, X, i, j - 1$ )
  
```

	0	1	2	3	4	5	6
	x_i	B	D	C	A	B	A
0 y_j	0	0	0	0	0	0	0
1 A	0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\nwarrow$ 1	$\leftarrow$ 1	$\nwarrow$ 1
2 B	0	$\nwarrow$ 1	$\nwarrow$ 1	$\leftarrow$ 1	$\uparrow$ 1	$\nwarrow$ 2	$\leftarrow$ 2
3 C	0	$\uparrow$ 1	$\uparrow$ 1	$\nwarrow$ 2	$\nwarrow$ 2	$\uparrow$ 2	$\uparrow$ 2
4 B	0	$\nwarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\leftarrow$ 3
5 D	0	$\uparrow$ 1	$\nwarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\uparrow$ 3
6 A	0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\uparrow$ 3	$\nwarrow$ 4
7 B	0	$\nwarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\nwarrow$ 4	$\uparrow$ 4

Output: BCBA



Uniqueness 最长公共子序列的唯一性

- This algorithm is deterministic. However, the LCS is not unique. Why? 两个字符串的最长公共子序列**不是唯一的**
- We set $b[i, j] \leftarrow \text{"}\uparrow\text{"}$ when $c[i - 1, j] = c[i, j - 1]$. However, $b[i, j] \leftarrow \text{"}\leftarrow\text{"}$ is also optimal in this case.

```
5 if  $x_i = y_j$  then
6    $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ 
7    $b[i, j] \leftarrow \text{"}\nwarrow\text{"}$ 
8   else if  $c[i - 1, j] \geq c[i, j - 1]$  then
9      $c[i, j] \leftarrow c[i - 1, j]$ 
10     $b[i, j] \leftarrow \text{"}\uparrow\text{"}$ 
11    else  $c[i, j] \leftarrow c[i, j - 1]$ 
12     $b[i, j] \leftarrow \text{"}\leftarrow\text{"}$ 
```



Uniqueness最长公共子序列的唯一性

		0	1	2	3	4	5	6
	x_i	B	D	C	A	B	A	
0	y_j	0	0	0	0	0	0	
1	A	0	$\uparrow 0$	$\uparrow 0$	$\uparrow 0$	$\swarrow 1$	$\swarrow 1$	
2	B	0	$\swarrow 1$	$\swarrow 1$	$\swarrow 1$	$\swarrow 2$	$\swarrow 2$	
3	C	0	$\uparrow 1$	$\uparrow 1$	$\swarrow 2$	$\uparrow 2$	$\uparrow 2$	
4	B	0	$\swarrow 1$	$\uparrow 1$	$\uparrow 2$	$\swarrow 3$	$\swarrow 3$	
5	D	0	$\uparrow 1$	$\swarrow 2$	$\uparrow 2$	$\uparrow 3$	$\uparrow 3$	
6	A	0	$\uparrow 1$	$\uparrow 2$	$\swarrow 3$	$\uparrow 3$	$\swarrow 4$	
7	B	0	$\swarrow 1$	$\uparrow 2$	$\uparrow 3$	$\swarrow 4$	$\uparrow 4$	

BCBA

BCAB

BDAB



Space Improvement 降低空间使用

- How each entry $c[i, j]$ is computed?
 - It depends only on $c[i - 1, j - 1]$, $c[i - 1, j]$, and $c[i, j - 1]$.
- If we **only need the length** of the LCS, we only need the row being computed and the previous row.
 - We can reduce the asymptotic space requirements by storing only these two rows. 如果只是需要知道LCS的长度的话，只需要两行的数组



Classroom Exercise

Draw the table of LCS of the following sequences, and find a LCS:

$$X = \langle A, D, C, A \rangle \quad Y = \langle A, C, D, A \rangle$$



Classroom Exercise

		0	1	2	3	4
		x_i	A	D	C	A
0	y_j	0	0	0	0	0
1	A	0				
2	C	0				
3	D	0				
4	A	0				



Classroom Exercise

		0	1	2	3	4
	x_i		A	D	C	A
0	y_j	0	0	0	0	0
1	A	0	$\nwarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\nwarrow 1$
2	C	0	$\uparrow 1$	$\uparrow 1$	$\nwarrow 2$	$\leftarrow 2$
3	D	0	$\uparrow 1$	$\nwarrow 2$	$\uparrow 2$	$\uparrow 2$
4	A	0	$\nwarrow 1$	$\uparrow 2$	$\uparrow 2$	$\nwarrow 3$

ACA



0/1 KNAPSACK PROBLEM

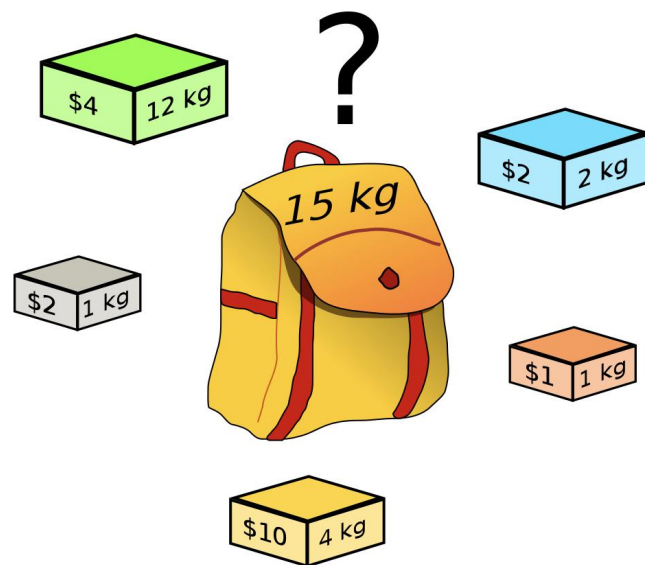
01 背包问题



0/1 Knapsack Problem

0/1 knapsack (0/1背包) problem:

- There are n items: the i th item is worth v_i dollars and weights w_i kg.
- n 个物品，第 i 个物品价值为 v_i 重量为 w_i ，总的背包的容量是 W 。目的是在不超过背包容量的前提下，使得背包里面的物品的总价值最高
- The capacity of knapsack is W kg.
- Items must be taken entirely or left behind.
- 一个物品不能被切割，要不放进去背包，要不就不放进去。
- Which items should we take to maximize the total value?





0/1 Knapsack Problem

- Mathematical description: Given an item set $s = \langle 1, 2, \dots, n \rangle$, and two n -tuples of positive numbers $\langle v_1, v_2, \dots, v_n \rangle$ and $\langle w_1, w_2, \dots, w_n \rangle$, and $W > 0$, we wish to determine the subset $s' \subseteq \{1, 2, \dots, n\}$ that
- 数学定义：找到S的子集S'，使得S'价值和最大，并且S'的重量和小于W

$$\text{maximize } \sum_{i \in s'} v_i$$

$$\text{subject to } \sum_{i \in s'} w_i \leq W$$



Example

- Weight capacity $W = 5\text{kg}$.
- The possible ways to fill the knapsack:
 - $\{1, 2, 3\}$ has value \$37 with weight 4kg.
 - $\{3, 4\}$ has value \$35 with weight 5kg.
 - $\{1, 2, 4\}$ has value \$42 with weight 5kg.
(optimal)

i	v_i	w_i
1	\$10	1kg
2	\$12	1kg
3	\$15	2kg
4	\$20	3kg



Optimal Substructure (最优子结构)

- The variables we should use in recursive equation must be the **maximum profit V** .
- 如何定义 V 的最优子结构
- How a V can be decomposed into the **V s** with smaller instance?
 - Item subset?
 - Smaller weight?



Optimal Substructure 最优子结构

- **Consider** the most valuable load that weights at most w kg.
- If item i is in the load and we remove it, the remaining load must be the most valuable load weighing at most $w - w_i$ that can be taken from the remaining $i - 1$ items.

假设我们找到了一个解法是最优的，假设我们把第 i 个物品从这个解法里面去除掉，那么对于容量为 $w - w_i$ 的背包，从1到 $i-1$ 的物品的装载方法也是最优的。

- Prove by contradiction: **if the remaining load is not the most valuable**, there exists a more valuable load and adding item i into it makes the original load not the most valuable.



Optimal Substructure

- $V[i, w]$: the maximum profit that can be obtained from items 1 to i , if the knapsack has size w . 当背包容量为 w 时，从1到 i 的物品中能够装的最大价值

- Case 1: take item i

$$V[i, w] = V[i - 1, w - w_i] + v_i$$

- Case 2: do not take item i

$$V[i, w] = V[i - 1, w]$$

- How to decide whether take item i or not?

- 判断下两种方式，到底哪个比较大

Simply compare and select the maximum one.



Recursive Equation

- The recursive equation:

$$V[i, w] = \begin{cases} V[i - 1, w] & w_i > w \\ \max \{V[i - 1, w], V[i - 1, w - w_i] + v_i\} & w_i \leq w \end{cases}$$

- $w_i > w$: we can't take w_i more than capacity w . 这个物品超过了背包的容量
- $w_i \leq w$: decide whether to take item i or not.



Additional Information

- $V[i, w]$ only records the optimal value, we need to record **additional information** to record the **items we take**.
- $b[i, w]$ records the **choices** made to obtain the optimal value.

- Case 1: take item i

$$b[i, w] = "\nwarrow"$$

- Case 2: do not take item i

$$b[i, w] = "\uparrow"$$



Filling Table

$$V[i, w] = \max \{V[i - 1, w], V[i - 1, w - w_i] + v_i\}$$

	0	1		$w - w_i$		w		W	
0	0	0	0	0	0	0	0	0	0
	0								
	0								
$i - 1$	0								
i	0								
	0								
n	0								



DPKnapsack(S, W)

```
1  for  $w \leftarrow 0$  to  $w_1 - 1$  do  $V[1, w] \leftarrow 0$ 
2  for  $w \leftarrow w_1$  to  $W$  do  $V[1, w] \leftarrow v_1$     (针对 $i=1$ , 一个物品的两种情况)
3  for  $i \leftarrow 2$  to  $n$  do
4      for  $w \leftarrow 0$  to  $W$  do
5          if  $w_i > w$  then
6               $V[i, w] \leftarrow V[i - 1, w]$ 
7               $b[i, w] \leftarrow " \uparrow "$ 
8          else if  $V[i - 1, w] > V[i - 1, w - w_i] + v_i$  then
9               $V[i, w] \leftarrow V[i - 1, w]$ 
10              $b[i, w] \leftarrow " \uparrow "$ 
11         else
12              $V[i, w] \leftarrow V[i - 1, w - w_i] + v_i$ 
13              $b[i, w] \leftarrow " \nwarrow "$ 
14  return  $V$  and  $b$ 
```

Running time: $\Theta(nW)$



$W = 5$

$$V[i, w] = \begin{cases} V[i-1, w] & w_i > w \\ \max\{V[i-1, w], V[i-1, w-w_i] + v_i\} & w_i \leq w \end{cases}$$

Item i	w_i	v_i
1	2	12
2	1	10
3	3	20
4	2	15

w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

$$V[1, 1] = V[0, 1] = 0$$

$$V[1, 2] = \max\{12 + 0, 0\} = 12$$

$$V[1, 3] = \max\{12 + 0, 0\} = 12$$

$$V[1, 4] = \max\{12 + 0, 0\} = 12$$

$$V[1, 5] = \max\{12 + 0, 0\} = 12$$

$$V[2, 1] = \max\{10 + 0, 0\} = 10$$

$$V[3, 1] = V[2, 1] = 10$$

$$V[4, 1] = V[3, 1] = 10$$

$$V[2, 2] = \max\{10 + 0, 12\} = 12$$

$$V[3, 2] = V[2, 2] = 12$$

$$V[4, 2] = \max\{15 + 0, 12\} = 15$$

$$V[2, 3] = \max\{10 + 12, 12\} = 22$$

$$V[3, 3] = \max\{20 + 0, 22\} = 22$$

$$V[4, 3] = \max\{15 + 10, 22\} = 25$$

$$V[2, 4] = \max\{10 + 12, 12\} = 22$$

$$V[3, 4] = \max\{20 + 10, 22\} = 30$$

$$V[4, 4] = \max\{15 + 12, 30\} = 30$$

$$V[2, 5] = \max\{10 + 12, 12\} = 22$$

$$V[4, 5] = \max\{20 + 12, 22\} = 32$$

$$V[4, 5] = \max\{15 + 22, 32\} = 37$$



Reconstructing the Optimal Solution

- Start at $V[n, W]$.
- When you go left-up, item i has been taken. 往左上角的时候，第 i 个物品有被装进背包
- When you go straight up, item i has not been taken. 往上走时，第 i 个物品没有被放进背包。

i	w					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

⑩ Item 4

⑩ Item 2

⑩ Item 1



Classroom Exercise

Fill the table to solve the following 0/1 knapsack problem when $W = 3$.

i	v_i	w_i
1	\$10	1kg
2	\$12	1kg
3	\$15	2kg



Classroom Exercise

$$V[i, w] = \begin{cases} V[i-1, w] & w_i > w \\ \max\{V[i-1, w], V[i-1, w-w_i] + v_i\} & w_i \leq w \end{cases}$$

Solution:

$W = 3$

i	v_i	w_i
1	\$10	1kg
2	\$12	1kg
3	\$15	2kg

		w			
		0	1	2	3
i	0	0	0	0	0
	1	0	10	10	10
	2	0	12	22	22
	3	0	12	22	27

$$V[1, 1] = \max\{10 + 0, 0\} = 10$$

$$V[1, 2] = \max\{10 + 0, 0\} = 10$$

$$V[1, 3] = \max\{10 + 0, 0\} = 10$$

$$V[2, 1] = \max\{10, 0 + 12\} = 12$$

•
•
•



编程作业

■ xmuoj动态规划

■ 矩阵链乘法问题

■ <http://xmuoj.com/problem/SFDP02>

■ 2: 01背包问题

■ <http://xmuoj.com/problem/ACW002>

- 要求提交代码cpp文件，以及word文档（问题1，问题2，是书上的内容，不需要分析，word里面要有截图，显示你的学号提交的代码accepted了。）



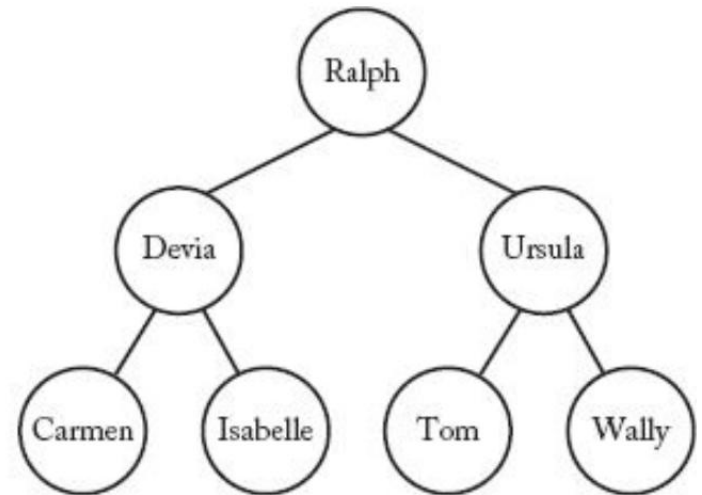
OPTIMAL BINARY SEARCH TREES

最优二叉搜索树



Optimal Binary Search Trees

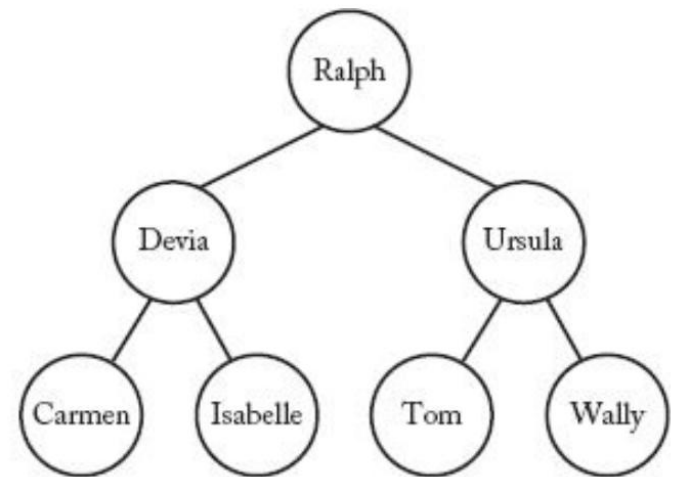
- A **binary search tree (BST)** (二叉搜索树) is a binary tree of keys that come from an ordered set, such that
 - Each node contains one key.
 - The keys in the **left** subtree of a given node are **less** than or equal to the key in that node.
 - 左子树的关键字比根节点的关键字小
 - 右子树的关键字比根节点的关键字大
 - The keys in the **right** subtree of a given node are **greater** than or equal to the key in that node.





Optimal Binary Search Trees

- The number of comparisons done by search to locate a key is called the **search time**. 在树上比较了几次，称作搜索时间
- We want to know the average search time of a BST while the keys **do not** have the same probability. 需要知道当树上的关键字的概率不是一样的话，那么平均搜索时间是多少
 - E.g. Tom is a common name in the United States. It has higher probability to be a search key.
 - Thus, put the node whose key has high probability to lower depth **will decrease the average search time**. 将有更高概率的节点放到比较低的深度位置



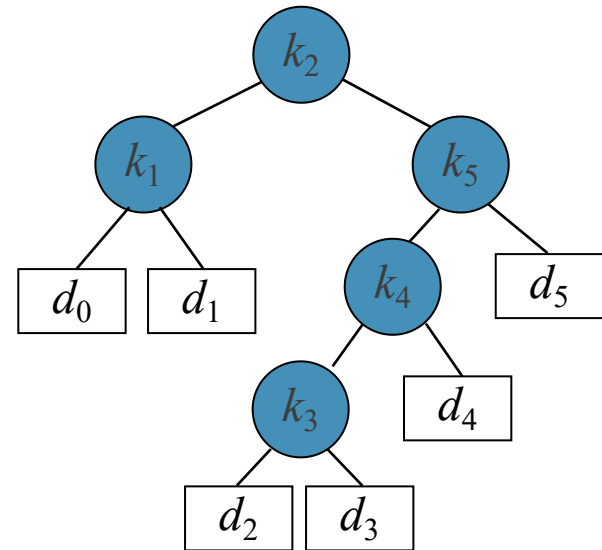
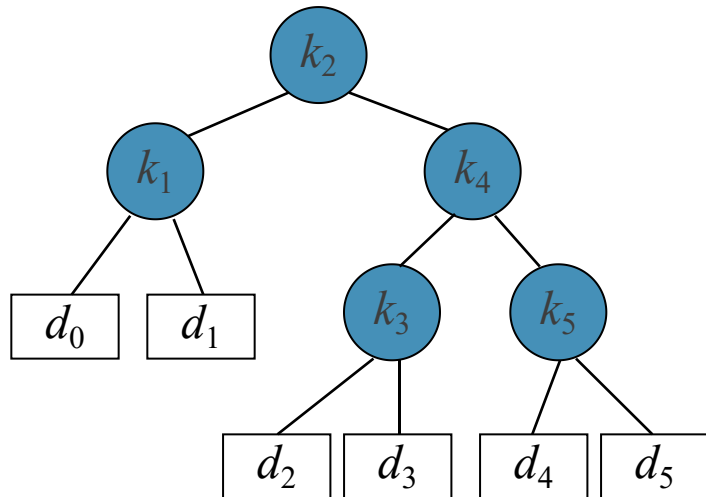


Optimal Binary Search Trees 最优搜索树

■ Given

- a **sequence** $K = \langle k_1, k_2, \dots, k_n \rangle$ of n distinct keys in sorted order (so that $k_1 < k_2 < \dots < k_n$) 假设这些关键字是从小到大的
- $n + 1$ “dummy keys (虚拟关键字)” $\langle d_0, d_1, d_2, \dots, d_n \rangle$ when the key is not in K , such that 虚拟关键字 表示的是一个区间

$$d_0 < k_1 < d_1 < k_2 < \dots < k_i < d_i < k_{i+1} < \dots < k_n < d_n$$





Optimal Binary Search Trees

- The search key x is **not uniformly distributed**. It follows the probability: 搜索树上关键字不是均匀分布的
 - For each key k_i , we have a probability p_i that a search will be for k_i .
关键字 k_i 的概率是 p_i
 - For each d_i , we have a probability q_i that a search will correspond to d_i . 对于区间 d_i ，一个关键字落到这个区间的概率是 q_i
- For each search, either some key k_i is found, or some dummy key d_i is found. Therefore, we have: 概率总和为1

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$

Where $\sum_{i=1}^n p_i$ is the probability for a **successful search** and $\sum_{i=1}^n q_i$ is the probability for a **failed search**.



Optimal Binary Search Trees

- Assume that **the actual cost** of a search is the number of **nodes examined**, i.e., the depth of the node found by the search in T , plus 1. 树的深度从0开始，所以要加1
- Then the expected cost of **a search in T** is

$$E(T) = \sum_{i=1}^n (d_T(k_i) + 1) \times p_i + \sum_{i=0}^n (d_T(d_i) + 1) \times q_i$$

where $d_T(k)$ is **the depth** of key k in tree T .

- For a given set of probabilities, our goal is to construct a BST whose $E(T)$ is **smallest**. This tree is called **optimal BST** (最优二叉搜索树).

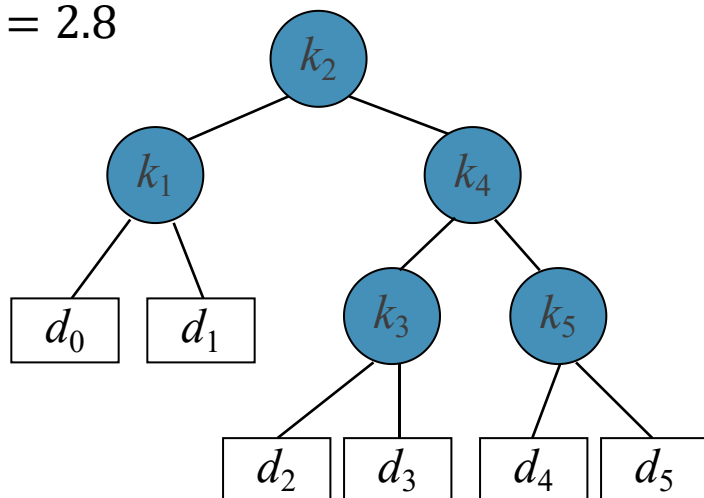


Example

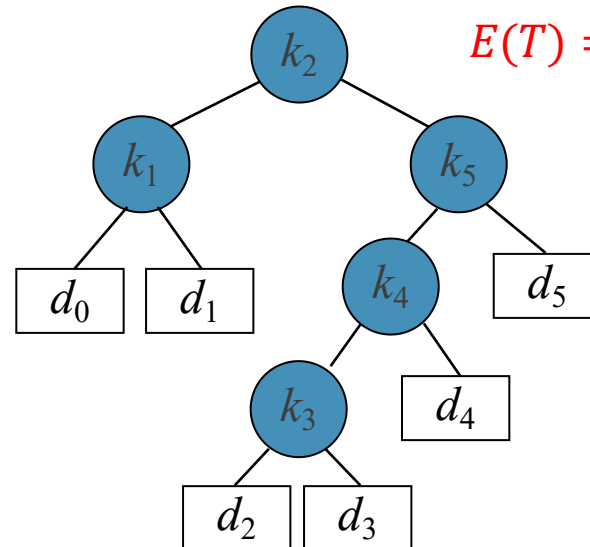
$$E(T) = \sum_{i=1}^n (d_T(k_i) + 1) \times p_i + \sum_{i=0}^n (d_T(d_i) + 1) \times q_i$$

i	0	1	2	3	4	5
p_i		0.15	0.1	0.05	0.1	0.2
q_i	0.05	0.1	0.05	0.05	0.05	0.1

$$E(T) = 2.8$$



$$E(T) = 2.75$$

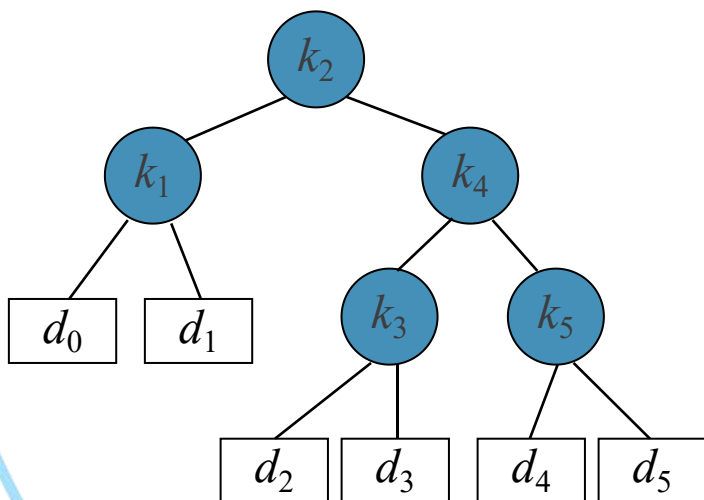




Example

$$E(T) = \sum_{i=1}^n (d_T(k_i) + 1) \times p_i + \sum_{i=0}^n (d_T(d_i) + 1) \times q_i$$

$$E(T) = 2.8$$

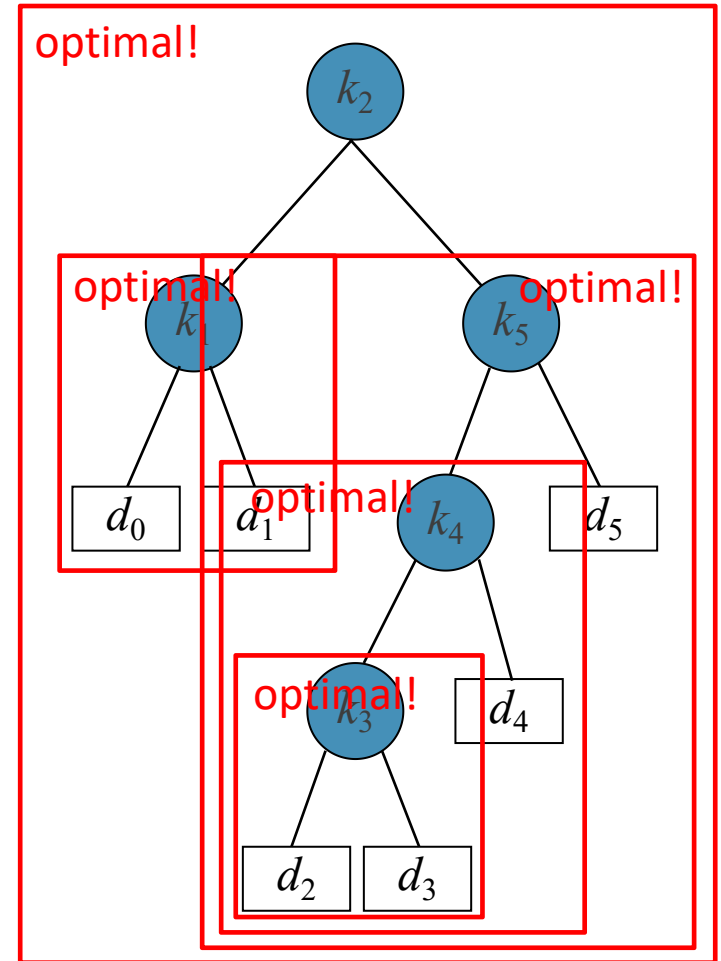


node	depth	probability	contribution
k_1	1	0.15	0.3
k_2	0	0.1	0.1
k_3	2	0.05	0.15
k_4	1	0.1	0.2
k_5	2	0.2	0.6
d_0	2	0.05	0.15
d_1	2	0.1	0.3
d_2	3	0.05	0.2
d_3	3	0.05	0.2
d_4	3	0.05	0.2
d_5	3	0.1	0.4



Optimal Substructure 最优子结构

- If a BST is optimal, all its subtrees are also optimal.
- Therefore, we should consider an arbitrary subtree of this problem.



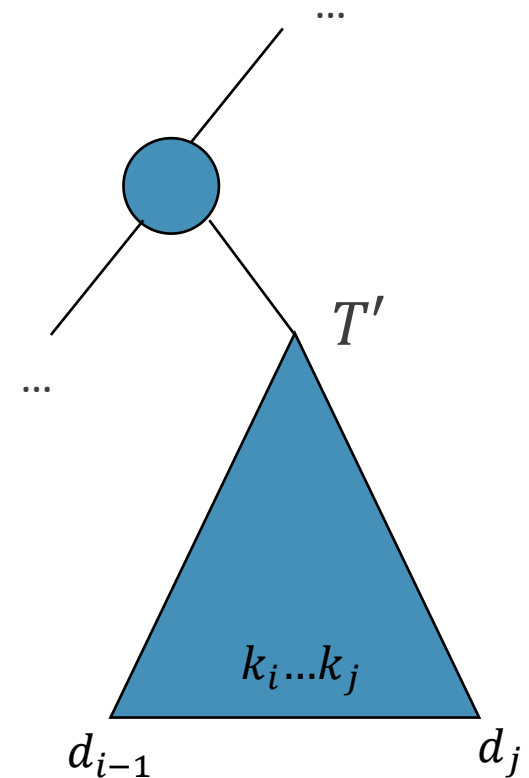


Optimal Substructure

- Consider any subtree T' of a BST (二叉搜索树的任意子树). It **must contain** keys in a **contiguous (连续的) range** $k_i, \dots, k_j$, for some $1 \leq i \leq j \leq n$.

任意一个子树，他的键值肯定是连续的

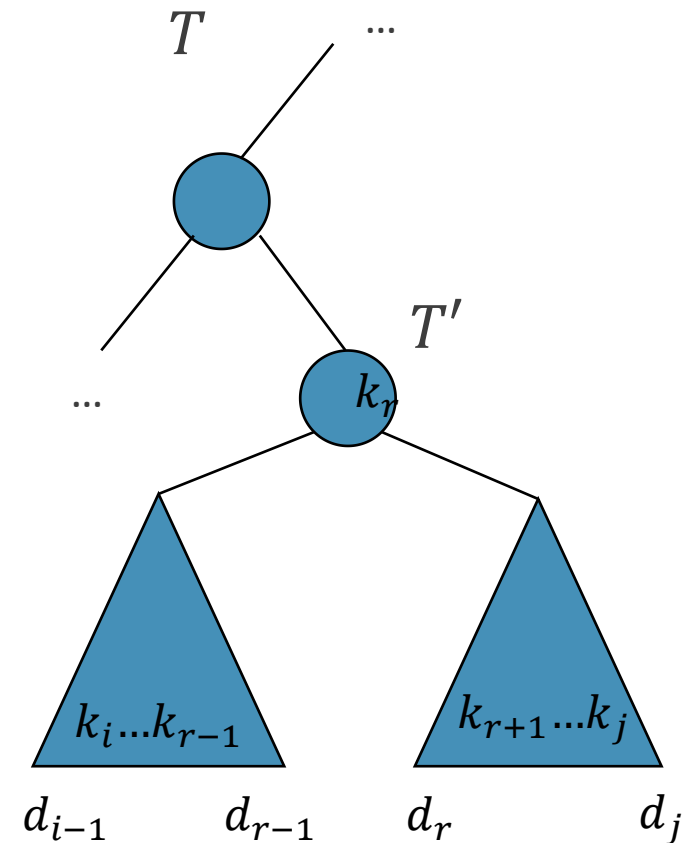
- In addition, a subtree that contains **keys** $k_i, \dots, k_j$ must also have as **its leaves** the dummy keys $d_{i-1}, \dots, d_j$.





Optimal Substructure

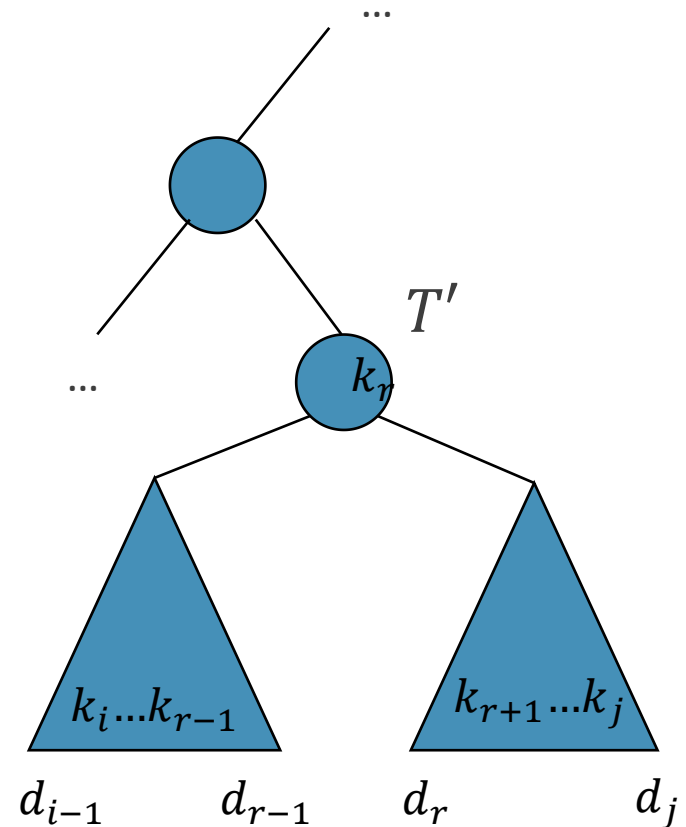
- If an optimal BST T has a subtree T' containing keys $k_i, \dots, k_j$, then this subtree T' must be optimal as well for the subproblem with keys $k_i, \dots, k_j$ and dummy keys $d_{i-1}, \dots, d_j$. 如果一个最优搜索树的子树包含了 k_i 到 k_j 的值, 那么这颗子树肯定是关于 $k_i, \dots, k_j$ 和 $d_{i-1}, \dots, d_j$ 的最优搜索树
- **Prove in contradiction.** If there were a subtree T'' whose expected cost is lower than that of T' , then we could cut T' out of T and paste in T'' , resulting in a BST of lower expected cost than T , thus contradicting the optimality of T .





Optimal Substructure 类似矩阵连乘的结构

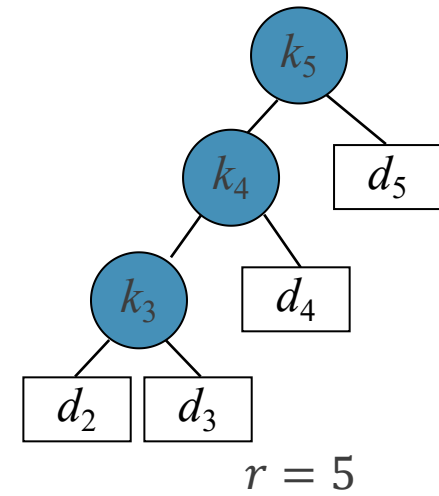
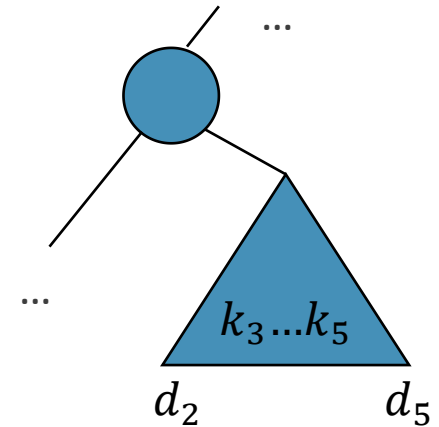
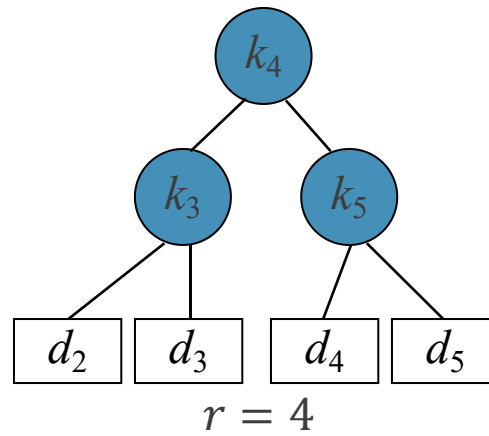
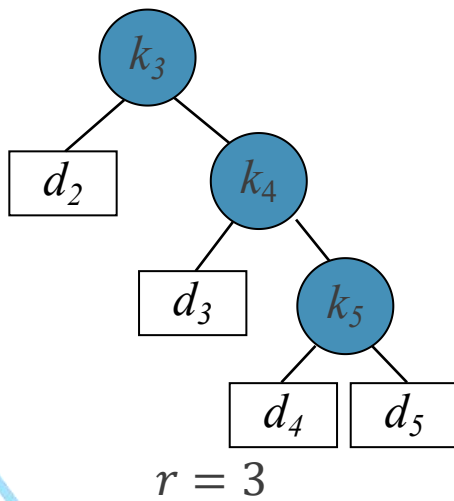
- The subtree T' must have a root k_r for $i \leq r \leq j$.
 - Left subtree of k_r contains $k_i, \dots, k_{r-1}$.
 - Right subtree of k_r contains $k_{r+1}, \dots, k_j$.
- Given contiguous keys $k_i, \dots, k_j$, **how to recursively find** an optimal subtree?
- Examine **all candidate roots k_r** , for $i \leq r \leq j$, and **select** the one with **minimal cost**. 变量所有可能作为根节点的 K_r





Optimal Substructure

- For example, **the subtree T'** has contiguous keys $k_3, \dots, k_5$ and dummy keys $d_2, \dots, d_5$.
- We construct **all the subtree cases** and **select** the one with **minimum** expected search time.





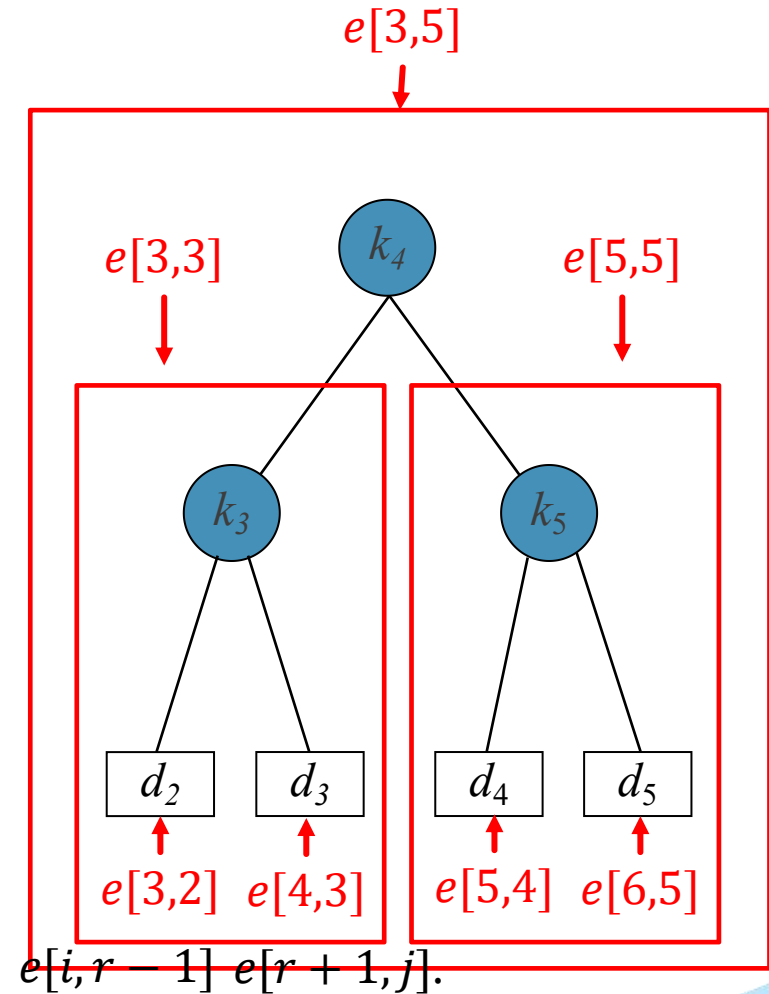
Recursive Equation 递归方程

- 定义 $e[i, j]$ 为 $k_i, \dots, k_j$ and dummy keys $d_{i-1}, \dots, d_j$ 的最优二叉搜索树的 **cost**
- Define $e[i, j]$ as expected search **cost** of **optimal BST** for $k_i, \dots, k_j$ and dummy keys $d_{i-1}, \dots, d_j$. 注意这边是 d_{i-1}
- If $j \geq i$, **select a root k_r** , for some $i \leq r \leq j$ and **recursively** make an optimal BST.
 - for $k_i, \dots, k_{r-1}$ as the left subtree, and
 - for $k_{r+1}, \dots, k_j$ as the right subtree.



Example

- If k_r is selected as the root of the subtree T' , $e[i, j]$ only relates to $e[i, r - 1]$ and $e[r + 1, j]$.
- If $j = i - 1$, the subtree contains no key but a single **dummy key**, then $e[i, j] = q_{i-1}$.



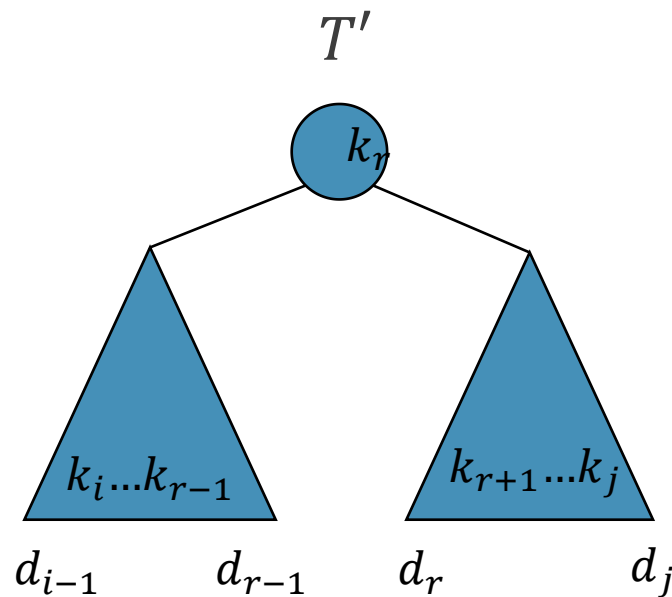


Recursive Equation

多了一层的树，要
额外加这些cost

- The total search cost is composed by **three parts**:
 - The search time for **the root**: p_r .
 - The search time for the left subtree: $e[i, r - 1] + \sum_{l=i}^{r-1} p_l + \sum_{l=i-1}^{r-1} q_l$.
 - The search time for the right subtree: $e[r + 1, j] + \sum_{l=r+1}^j p_l + \sum_{l=r}^j q_l$.
- Let $w[i, j] = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$, the total is:
 $e[i, j] = e[i, r - 1] + e[r + 1, j] + w[i, j]$.

Because the subtrees have one more depth, we should add the probabilities of all their keys and dummy keys.





Example

i	0	1	2	3	4	5
p_i		0.15	0.1	0.05	0.1	0.2
q_i	0.05	0.1	0.05	0.05	0.05	0.1

$$e[3,3] = 1 \times p_3 + 2 \times (q_2 + q_3)$$

$$e[5,5] = 1 \times p_5 + 2 \times (q_4 + q_5)$$

$$e[3,5] = 1 \times p_4$$

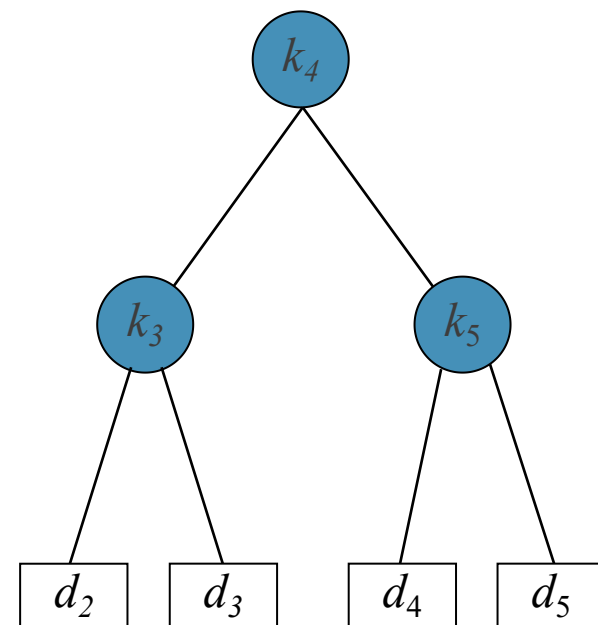
$$+ (1 + 1) \times p_3 + (2 + 1) \times (q_2 + q_3)$$

$$+ (1 + 1) \times p_5 + (2 + 1) \times (q_4 + q_5)$$

$$= e[3,3] + e[5,5]$$

$$+ \sum_{l=3}^5 p_l + \sum_{l=2}^5 q_l$$

$$= e[3,3] + e[5,5] + w[3,5]$$





Recursive Equation

- We **iterate over all** k_r and select the one with **minimal cost**:

$$e[i, j] = \begin{cases} q_{i-1} & j = i - 1 \\ \min_{i \leq r \leq j} \{e[i, r - 1] + e[r + 1, j] + w[i, j]\} & i \leq j \end{cases}$$

- To avoid repeated computation, we can also **recursively calculate** $w[i, j] = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$ 关键字和虚拟关键字的概率和

$$w[i, j] = \begin{cases} q_{i-1} & j = i - 1 \\ w[i, j - 1] + p_j + q_j & 1 \leq i \leq j \leq n \end{cases}$$

- Both $e[i, j]$ and $w[i, j]$ are tables with $i = 1, \dots, n + 1, j = 0, \dots, n$.



填表过程和矩阵连乘类似

$$e[i, j] = \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w[i, j]\}$$

i

	1	2	3	4	5	6
5						
4		$e[i, j]$	$e[i+1, j]$	...	$e[j+1, j]$	
3		$e[i, j-1]$				
2		...				
1		$e[i, i-1]$				
0						

$$w[i, j] = w[i, j-1] + p_j + q_j$$

i	1	2	3	4	5	6
5						
4		$w[i, j]$				
3		$w[i, j-1]$				
2						
1						
0						



Pseudocode (和矩阵连乘类似)

```
DPOptimalBST( $p, q, n$ )
1 for  $i \leftarrow 1$  to  $n + 1$  do
2    $e[i, i - 1] \leftarrow q_{i-1}$ 
3    $w[i, i - 1] \leftarrow q_{i-1}$ 
4 for  $c \leftarrow 1$  to  $n$  do
5   for  $i \leftarrow 1$  to  $n - c + 1$  do
6      $j \leftarrow i + c - 1$ 
7      $e[i, j] \leftarrow \infty$ 
8      $w[i, j] \leftarrow w[i, j - 1] + p_j + q_j$ 
9     for  $r \leftarrow i$  to  $j$  do
10       $t \leftarrow e[i, r - 1] + e[r + 1, j] + w[i, j]$ 
11      if  $t < e[i, j]$  then
12         $e[i, j] \leftarrow t$ 
13         $root[i, j] \leftarrow r$ 
14 return  $e$  and  $root$ 
```

First diagonal

From 1st to n th diagonal

Given c , determine the j th column and i th row

和矩阵连乘只
有一点点区别

Running time: $\Theta(n^3)$



$$e[i, j] = \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w[i, j]\}$$

		1	2	3	i	4	5	6
5		2.75	2.0	1.3		0.9	0.5	0.1
4		1.75	1.2	0.6		0.3	0.05	
3		1.25	0.7	0.25		0.05		
2		0.9	0.4	0.05				
1		0.45	0.1					
0		0.05						
	j	e						

i	0	1	2	3	4	5
p_i		0.15	0.1	0.05	0.1	0.2
q_i	0.05	0.1	0.05	0.05	0.05	0.1

$$w[i, j] = w[i, j-1] + p_j + q_j$$

		1	2	3	i	4	5	6
5		1	0.8	0.6		0.5	0.35	0.1
4		0.7	0.5	0.3		0.2	0.05	
3		0.55	0.35	0.15		0.05		
2		0.45	0.25	0.05				
1		0.3	0.1					
0		0.05						
	j	w						

		1	2	3	4	5
5		2	4	5	5	5
4		2	2	4	4	
3		2	2	3		
2		1	2			
1		1				
	j	i				

root



Classroom Exercise 课堂练习

Find the optimal BST with the following key probabilities:

i	0	1	2	3
p_i		0.25	0.3	0.15
q_i	0.05	0.1	0.05	0.1


$$e[i, j] = \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w[i, j]\} \quad w[i, j] = w[i, j-1] + p_j + q_j$$

i

	1	2	3
3			3
j 2		2	
1	1		

$root$



Classroom Exercise

i	0	1	2	3
p_i		0.25	0.3	0.15
q_i	0.05	0.1	0.05	0.1

$$e[i, j] = \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w[i, j]\}$$

$$w[i, j] = w[i, j-1] + p_j + q_j$$

		1	2	i	3	4
3		2	1.25	0.45	0.1	
2		1.35	0.6	0.05		
1		0.55	0.1			
0		0.05				
	j					
		e				

		1	2	i	3	4
3		1	0.7	0.3	0.1	
2		0.75	0.45	0.05		
1		0.4	0.1			
0		0.05				
	j					
		w				

			i	
		1	2	3
3		2	3	3
2		1	2	
1		1		
	j			
		$root$		



ELEMENTS OF DYNAMIC PROGRAMMING

动态规划的基本性质



动态规划两个主要性质

最优化问题能用动态规划求解的基本条件

- 最优子结构
 - 最优解所包含的子问题的解也是子问题的最优解
- 重叠子问题
 - 问题求解过程中，会有大量同样的子问题被求解（调用）



Optimal Substructure 最优子结构

找最优子结构规律：

- The solution to the problem consists of **making a choice**. Making this choice leaves one or more subproblems to be solved. 问题的解决涉及到做出决策，做出决策之后导致有若干子问题需要解决
- For a given problem, you are given the choice that leads to an optimal solution. 假定一个导致最优解的决策，先不关心如何得到这个决策
- Given this choice, you determine which subproblems follow and how to best characterize the resulting space of subproblems. 给定这么一个最优决策，确定所划分的子问题，以及如何方便分析问题空间的变化
- Solutions to the subproblems used within the optimal solution to the problem must themselves be optimal by proving by contradiction. 在问题最优解中用到的子问题的最优解，一定是子问题的最优解，可以用反证法



Optimal Substructure

最优子结构涉及以下两个问题：

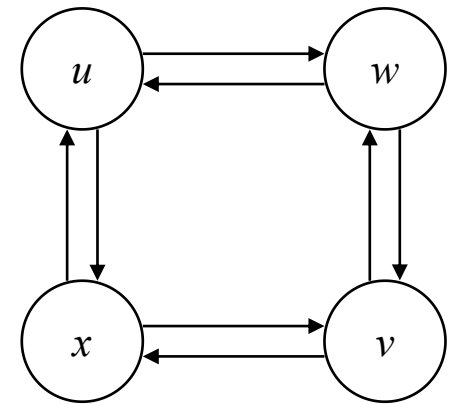
Optimal substructure varies across problem domains in **two ways**:

1. How many **subproblems** are used **in an optimal solution** to the original problem. 切成多少子问题
 - Assembly line: One subproblem ($f_1[j - 1]$ or $f_2[j - 1]$)
 - Matrix multiplication: Two subproblems (subproducts $A_{i...k}$ and $A_{k+1...j}$)
2. How many choices we have in determining which subproblem(s) to use in an optimal solution. 有多少不同的方式切分子问题从而得到最优决策
 - Assembly line: Two choices (line 1 or line 2)
 - Matrix multiplication: $j - i$ choices for k (splitting the product)



Optimal Substructure

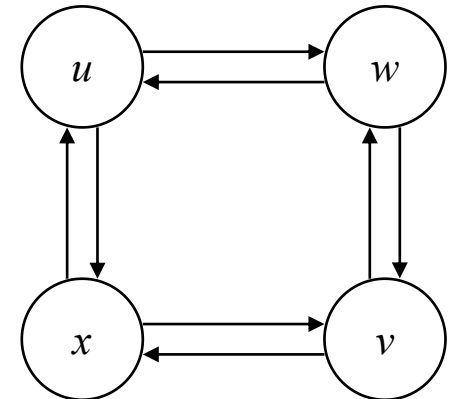
- Dynamic programming uses **optimal substructure** in a bottom-up fashion. 动态规划是自底向上的
- One should be careful **not to assume that optimal substructure applies when it does not.** 有的子结构不是最优的，就不能假设他是最优的
- Consider the following two problems in which we are given a directed graph $G = (V, E)$ and vertices $u, v \in V$.





Optimal Substructure

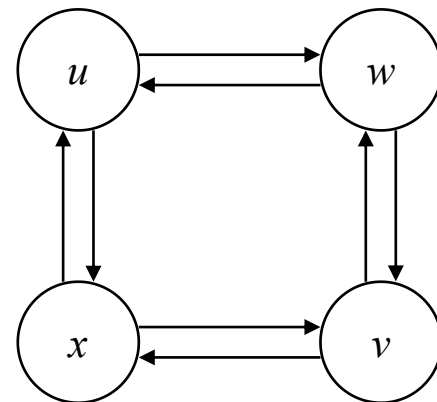
- **Unweighted shortest path:** Find a path from u to v consisting of the fewest edges. 不加权的最短路径问题
 - Such a path must be **simple**, since removing a cycle from a path produces a path with fewer edges.
- This problem **has optimal substructure**.
 - Assume the shortest path from u to v goes through w . Then $u \rightarrow w$ and $w \rightarrow v$ is also the shortest.





Optimal Substructure

- **Unweighted longest simple path:** Find a **simple** path from u to v consisting of the most edges. 不加权的最长路径（不带环）问题
 - Simplicity（必须是简单路径）is necessary because otherwise we can traverse a cycle（简单路径不带回路）as many times as we like to create paths with an arbitrarily large number of edges.
- For unweighted longest path, the problem **does not have** optimal substructure. 这个问题不是有最优子结构的
 - Assume the longest path from u to v goes through w . Then $u \rightarrow w$ may not be the longest. $u \rightarrow x \rightarrow v \rightarrow w$ is the longest.



解各子问题，互相独立，是问题具有最优子结构性质的关键



动态规划-编程

- 编写递归方程，原问题变成若干个子问题
- 定义数组，确定数据的每个元素属于什么意义
- 确定初始条件
- 确定边界条件
- 填表



Memorization (备忘录)

- **Memorization (备忘录) method** still uses recursion but store values in the table after calculation. 备忘录方法是一种非常有效的降低编程复杂度的方法，可以节约在编填表(Fill Table)程序上的时间

```
LookUpChain(p, i, j)
1  if  $m[i, j] < \infty$  then return  $m[i, j]$ 
2  if  $i = j$  then
3       $m[i, j] \leftarrow 0$ 
4  else for  $k \leftarrow i$  to  $j - 1$  do
5       $q \leftarrow \text{LookUpChain}(p, i, k) +$ 
            $\text{LookUpChain}(p, k + 1, j) + p_{i+1}p_kp_j$ 
6      if  $q < m[i, j]$  then
7           $m[i, j] \leftarrow q$ 
8  return  $m[i, j]$ 
```

```
MemoizedMatrixChain(p)
1  for  $i \leftarrow 1$  to  $n$  do
2      for  $j \leftarrow i$  to  $n$  do
3           $m[i, j] \leftarrow \infty$ 
4  return LookUpChain(p, 1, n)
```

Easier to implement because no diagonal trick is needed, but more recursive calls are required. 在矩阵连乘这个例子中，显得非常简单，因为不需要编对角线的技巧



Dynamic Programming vs. Memorization

■ Advantages of dynamic programming

- No overhead (系统开销) for recursion, less overhead for maintaining the table. 自底向上的方式，动态规划节约了不少用在递归上的overhead
- The regular pattern of table accesses may be used to reduce time or space requirements. 采用表格法的方式能够有效降低空间和时间的开销

■ Advantages of memorization 备忘录法的优点

- 编程起来非常方便，解决了花在编写自底向上填表的时间

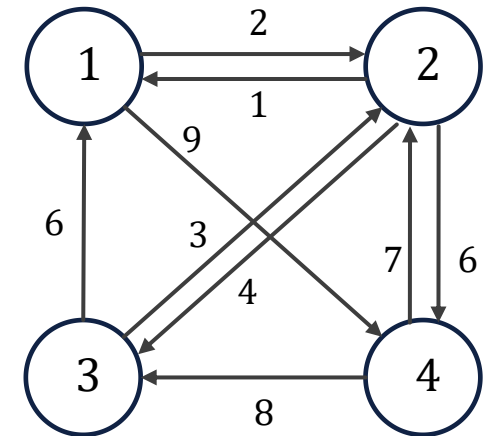


TRAVELING SALESPERSON PROBLEM



Traveling Salesperson Problem

- Given an n vertex network (undirected or directed), **traveling salesperson problem** (旅行商问题, TSP) is to find **a cycle of minimum cost** that includes all n vertices.
 - Hamiltonian cycle with minimum cost.
- Any cycle that includes all n vertices of a network is called a tour. In TSP, we are to find a least-cost tour. For example:
 - Tour (1,2,3,4,1) costs 22.
 - Tour (1,3,2,4,1) costs 26.
 - Tour (1,3,4,2,1) costs 21, optimal.





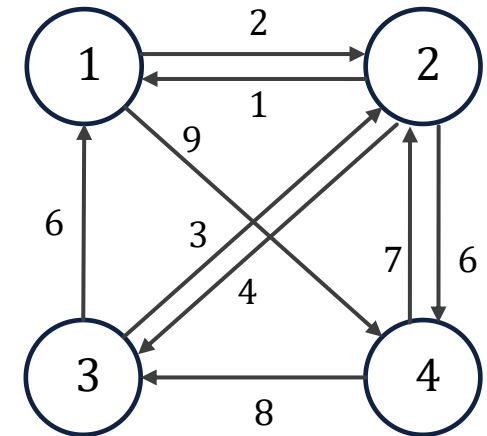
Traveling Salesperson Problem





Optimal Substructure

- Let v_k be the first vertex after v_1 on **an optimal tour**.
- The subpath of that tour **from v_k to v_1 must be a shortest path** that passes through each of the other vertices exactly once.
- If $k = 3$ and tour $(1,3,4,2,1)$ is optimal, then the subpath $(3,4,2,1)$ must be optimal.
 - Otherwise, we choose $(3,2,4,1)$ for better solution.





Optimal Substructure

- Let $V = \{1, \dots, n\}$ the set of all vertex indices, and $A \subseteq V$.
- Denote $D[i][A]$ the length of the shortest path from v_i to v_1 passing through each vertex in A exactly once. 从节点*i*通过*A*中节点（每个节点只有一次）到达节点1的最短路径长度
- We get the optimal tour:

$$\min_{2 \leq j \leq n} (w[1, j] + D[j][V - \{1, j\}])$$



Optimal Substructure

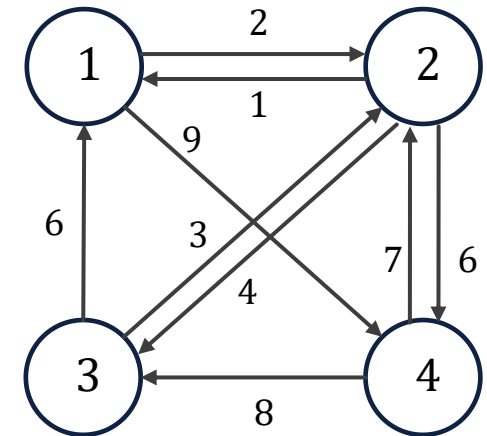
- Optimal tour:

$$\min_{2 \leq j \leq n} (w[1, j] + D[j][V - \{1, j\}])$$

- In this example, optimal tour is calculated by:

$$\min \begin{pmatrix} w[1, 2] + D[2][\{3, 4\}] \\ w[1, 3] + D[3][\{2, 4\}] \\ w[1, 4] + D[4][\{2, 3\}] \end{pmatrix}$$

- The rest problem is to recursively calculate $D[i][A]$.





Recursive Equation

- The recursive equation is:

$$D[i][A] = \begin{cases} \min_{j \in A} (w[i, j] + D[j][A - \{j\}]) & i \notin A \\ \infty & i \in A \end{cases}$$
$$D[i][\emptyset] = w[i, 1]$$

- The optimal tour is then $D[1][V - \{1\}]$.
- $D[j][A - \{j\}]$ is calculated in bottom-up fashion.



Recursive Equation

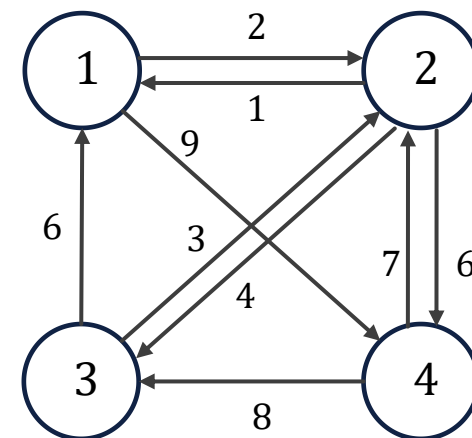
- In this example, optimal tour is calculated by:

$$\min \begin{pmatrix} w[1,2] + D[2][\{3,4\}] \\ w[1,3] + D[3][\{2,4\}] \\ w[1,4] + D[4][\{2,3\}] \end{pmatrix}$$

- Then,

$$D[2][\{3,4\}] = \min (w[2,3] + D[3][\{4\}])$$

$$\begin{aligned} D[3][\{4\}] &= \min (w[3,4] + D[4][\emptyset]) \\ &= (w[3,4] + w[4,1]) \end{aligned}$$





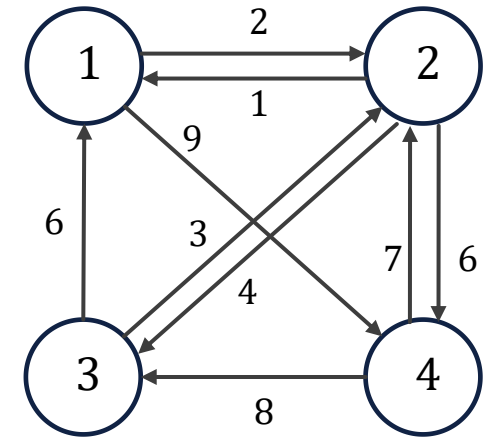
Filling Table

- The recursive equation is:

$$D[i][A] = \min_{j \in A} (w[i, j] + D[j][A - \{j\}])$$

- Then, the table can be built:

A



	$\emptyset$	$\{2\}$	$\{3\}$	$\{4\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{2,3,4\}$
1								
2								
3								
4								

i

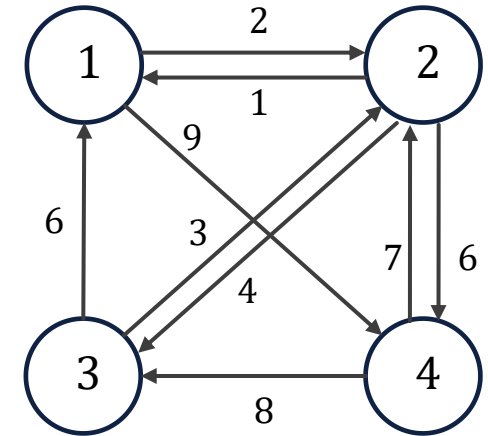


Filling Table

- The recursive equation is:

$$D[i][A] = \min_{j \in A} (w[i,j] + D[j][A - \{j\}])$$

- Then, the table can be built:



A

	$\emptyset$	$\{2\}$	$\{3\}$	$\{4\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{2,3,4\}$
1	—	—	—	—	—	—	—	21
2	1	∞	10	∞	∞	∞	20	—
3	6	4	∞	∞	∞	∞	∞	—
4	∞	8	14	∞	12	∞	∞	—

$$\min (w[4,2] + D[2][\{3\}], w[4,3] + D[3][\{2\}]) = \min (17, 12)$$



Filling Table

- There is only one remaining problem:

A

i	A							
	$\emptyset$	$\{2\}$	$\{3\}$	$\{4\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{2,3,4\}$
	1							
	2							
	3							
	4							

How to represent the subsets?



State Compression Dynamic Programming (状态DP)

- We can use binary coding for the subsets.
- This technique is called **state compression (状态压缩)**.

$\emptyset$	{2}	{3}	{4}	{2,3}	{2,4}	{3,4}	{2,3,4}
-------------	-----	-----	-----	-------	-------	-------	---------

如果一个城市被经过了用1表示，没有到过用0表示一个整数就可以表示所有城市是否被走过，那么可以得到 $dp[i][j]$ 其中i表示当前经过和没有经过的（二进制），j表示你站在j城市。所以方程得到：

$$dp[i][j] = \min_{1 \leq k < j} \{ dp[i \oplus (1 \ll (j-1))][k] + w[j] \}$$

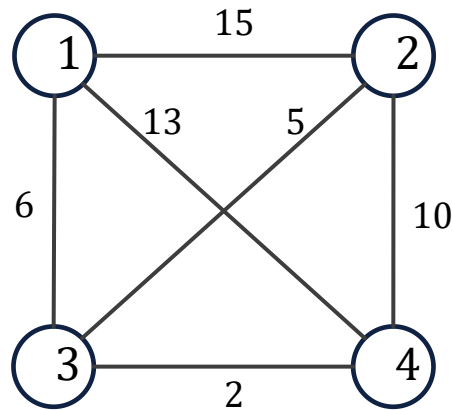
• $i \oplus (1 \ll (j-1))$;

• 这个运算一般用来给集合赋值。比如把i的二进制从右第j-1个改成1（不管是不是1都改）



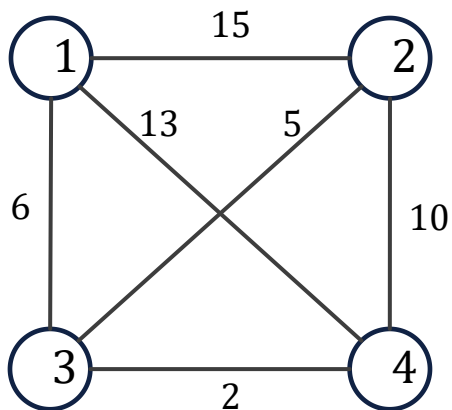
Classroom Exercise

- Fill in the table for this TSP instance.





Classroom Exercise



A

i

	$\emptyset$	$\{2\}$	$\{3\}$	$\{4\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{2,3,4\}$
1	—	—	—	—	—	—	—	33
2	15	∞	11	23	∞	∞	18	—
3	6	20	∞	15	∞	27	∞	—
4	13	25	8	∞	21	∞	∞	—



Conclusion

After this lecture, you should know:

- The difference between divide-and-conquer and dynamic programming.
- Why is dynamic programming efficient.
- What is optimal substructure.
- The steps of designing a dynamic programming algorithm.

分治算法更适合那些子问题独立且问题规模容易缩小到容易解决的程度的情况。动态规划则适用于子问题之间存在重叠，且可以通过递归方程逐步构建出整个问题解的情况。



课后习题

■ Page 92-94

6.2

6.3

6.7

6.9



谢谢

有问题欢迎随时跟我讨论



Example (LeetCode 62)



- 一个机器人，在地图的左上角位置，要走到地图的左下角星星位置，机器人只能往右走和往下走，一次走一个格子，请问一共有多少种走的方式
- 定义数据结构 $f[i][j]$ 代表走到第 i 行 j 列一共有多少种走法
- $f[i][j] = f[i-1][j] + f[i][j-1]$ 递归方程
- 边界条件 $i=0$ 以及 $j=0$ 时
- 初始条件 $f[0][0] = 1$
- 填表方式从上到下，从左到右



- 2 : 矩阵链乘法问题
- <http://xmuoj.com/problem/SFDP02>



Classroom Exercise

游戏绝地求生中有头盔和防弹衣, 分别分为3个等级:

- 头盔: 一级头, 二级头, 三级头.
- 护甲: 一级甲, 二级甲, 三级甲.

在游戏中一开始什么都没有, 但是会在路上随机捡取装备. 假设规定高级装备可以覆盖低级装备, 也可以直接穿上, 但是穿上高级装备后就无法再穿回低级装备. 那么从什么都没有, 到穿上三级头和三级甲, 一共有多少种方式?

例如

- 一级头->二级甲->三级甲->三级头
- 三级头->三级甲
- ...





Classroom Exercise

- 如果只考虑头盔, 那么穿上 i 级头有 2^{i-1} 种方式. 因为 i 级头一定要穿上, 但是 $1, \dots, i-1$ 级头都可以选择穿与不穿.
- 当穿上 i 级头和 j 级甲时, 可以分解为两种情况:
 - 已经穿上 j 级甲了, 就差 i 级头了, 这时候捡到 i 级头了.
 - 已经穿上 i 级头了, 就差 j 级甲了, 这时候捡到 j 级甲了.
- 因此, 假设 $H(i, j)$ 为穿上 i 级头和 j 级甲的方式数, 该问题的递归方程可以写为:

$$H(i, j) = \begin{cases} 2^{i-1} & i = 0 \text{ or } j = 0 \\ \sum_{k < i} H(k, j) + \sum_{k < j} H(i, k) & i > 0, j > 0 \end{cases}$$